

912 历年真题解析（2016-2023 版本）-数据结构篇

为了方便,本解析把排序问题放到了最后,按照绪论,向量,列表,栈和队列,树,图,搜索树,高级搜索树,散列,优先级队列,串,排序,算法题.

另外自己看书或者习题解析的时候,一定要对照勘误表: [DSACPP \(tsinghua.edu.cn\)](https://dsacpp.tsinghua.edu.cn)

我总结的可能会考的知识点不是说一定会考,毕竟 912 没有考纲的.

另外,所有题目本身和我所写的解析“都有可能出错”

绪论

1. 指针 p 指向某一个逻辑地址,那么 $p++$ 就是访问下一个逻辑地址 (×) (2016)

解析: 题目有些模糊,我理解的题意比如对于列表来说 $p++$ 并不一定是下一个节点的地址所以认为是错了,也说不清是哪里的知识点,暂时放在绪论里.

❖ 某些语言不支持指针类型,即便支持

频繁的动态空间分配也影响总体效率

❖ 此时,又当如何实现列表结构呢?

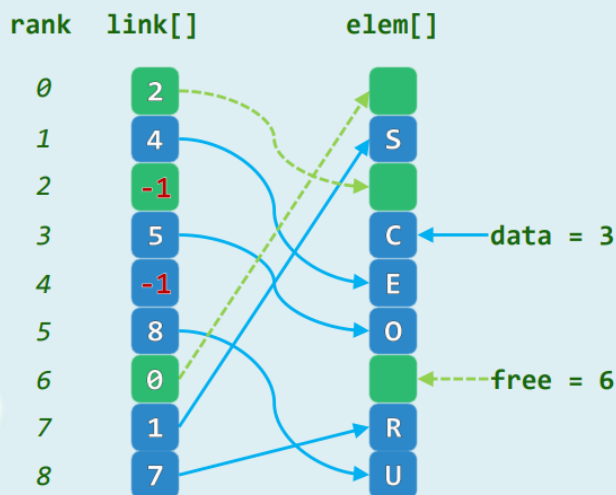
❖ 利用线性数组,以游标方式模拟列表

- $elem[]$: 对外可见的数据项

- $link[]$: 数据项之间的引用

❖ 维护逻辑上互补的列表 $data$ 和 $free$

❖ 在插入或删除元素时,应如何调整?



语言 C++ 编译器 g++ 版本 5.4.0

运行(Run)

```
1 #include <stdio.h>
2 #include <iostream>
3 //比如说这个elem[]是列表的游标实现中的数据项对应的数组
4 //p指向elem[0],p++只是指向elem[1]的并不一定是elem[0]的next
5 int main() {
6     using namespace std;
7     int elem[5] = {1,2,3,4,5};
8     int *p = elem;
9     cout << *p << endl;
10    p++;
11    cout << *p << endl;
12    return 0;
13 }
```

1
2

2. 若时间复杂度 $f(n) = O(g(n))$, 也不一定有 $f(n) = O(g(n-1))$ (√) (2017)

解析: 注意到: $f(n) = g(n) = n^n \neq (n-1)^{n-1}$, 所以本题答案是判断为对, 又如

$$f(n) = g(n) = \begin{cases} n, & n \text{ 是偶数} \\ n^2, & n \text{ 是奇数} \end{cases}, \text{ 则 } g(n-1) = \begin{cases} n-1, & n \text{ 是奇数} \\ (n-1)^2, & n \text{ 是偶数} \end{cases}$$

3. $T(1) = a$, 无论常数 a 多大, 时间复杂度为 $T(n) = T\left(\frac{n}{2}\right) + O(1)$ 的解总是 $O(\log n)$ (✓) (2018)

解析: 用主定理秒了 (见第 8 题)

4. 就地算法是指空间复杂度() (2018)

(A) $O(1)$ (B) $O(n)$ (C) $O(n^2)$ (D) $O(n \log n)$

解析: 基本常识

5. $n^{\lg \lg n} = O([\lg n]!)$ (✓) (2019)

解析: 注意到 $2^{\lg(n^{\lg \lg n})} = 2^{\lg \lg n \cdot \lg n} = 2^{\lg(\lg n) \lg n} = (\lg n)^{\lg n}$

原命题 $\Leftrightarrow (\lg n)^{\lg n} = O([\lg n]!) \Leftrightarrow (\lg M)^M = O(M!)$. (不妨设 $n = 2^M$)

两边取对数 $M \lg M = O(M \lg M)$ 成立, 因此原命题成立.

6. $\lg^n n = \Theta(n^{\lg n})$ (✗) (2020)

解析: $\lg^n n = 2^{\lg(\lg^n n)} = 2^{n \lg \lg n}, n^{\lg n} = 2^{\lg(n^{\lg n})} = 2^{\lg^2 n}$

7. 考查复杂度函数 $f(n) = 2^{\lg^2 n}$ 和 $g(n) = n^{2020}$, 从渐进的角度看, 何者增长更快? 为什么? (2021, 5 分)

解析: 注意到 $\lg f(n) = \lg^2 n$, 而 $\lg g(n) = 2020 \lg n$

当 n 充分大时, $\lg^2 n \gg 2020 \lg n$, 因此从渐进的角度 $f(n)$ 增长的更快。

8. 若某复杂度函数 $f(n)$ 满足: $f(0) = f(1) = O(1)$, 且对于 $n \geq 2$ 满足递推式: $f(n) = 3f\left(\frac{n}{9}\right) + O(1)$, 则就大 O 记号而言, $f(n)$ 属于哪个渐进复杂度层次? 为什么? (2021, 6 分)

解析: 这个用主定理可以直接秒了的, 推导过程如下

Master Theorem

- ❖ 分治策略对应的递推式通常 (尽管不总是) 形如 : $T(n) = a \cdot T(n/b) + \mathcal{O}(f(n))$
(原问题被分为 a 个规模均为 n/b 的子任务 ; 任务的划分、解的合并耗时 $f(n)$)
- ❖ 若 $f(n) = \mathcal{O}(n^{\log_b a - \epsilon})$, 则 $T(n) = \Theta(n^{\log_b a})$
 - kd-search: $T(n) = 2 \cdot T(n/4) + \mathcal{O}(1) = \mathcal{O}(\sqrt{n})$
- ❖ 若 $f(n) = \Theta(n^{\log_b a} \cdot \log^k n)$, 则 $T(n) = \Theta(n^{\log_b a} \cdot \log^{k+1} n)$
 - binary search: $T(n) = 1 \cdot T(n/2) + \mathcal{O}(1) = \mathcal{O}(\log n)$
 - mergesort: $T(n) = 2 \cdot T(n/2) + \mathcal{O}(n) = \mathcal{O}(n \cdot \log n)$
 - STL mergesort: $T(n) = 2 \cdot T(n/2) + \mathcal{O}(n \cdot \log n) = \mathcal{O}(n \cdot \log^2 n)$
- ❖ 若 $f(n) = \Omega(n^{\log_b a + \epsilon})$, 则 $T(n) = \Theta(f(n))$
 - quickSelect (average case): $T(n) = 1 \cdot T(n/2) + \mathcal{O}(n) = \mathcal{O}(n)$

$$\begin{aligned}
 f(n) &= 3f\left(\frac{n}{9}\right) + \mathcal{O}(1) \\
 3f\left(\frac{n}{9}\right) &= 3^2 f\left(\frac{n}{9^2}\right) + 3\mathcal{O}(1) \\
 3^2 f\left(\frac{n}{9^2}\right) &= 3^3 f\left(\frac{n}{9^3}\right) + 3^2 \mathcal{O}(1) \\
 &\vdots \\
 3^k f\left(\frac{n}{9^k}\right) &= 3^{k+1} f(1) + 3^k \mathcal{O}(1),
 \end{aligned}$$

其中 $9^{k+1} = n \Rightarrow 3^{k+1} = \sqrt{n}$

$$\text{因此 } f(n) = 3^{k+1} f(1) + (1+3+3^2+\dots+3^k) \mathcal{O}(1) = \frac{1}{2}(3^{k+2}-1) \mathcal{O}(1) = \mathcal{O}(\sqrt{n})$$

9. $[\log n]! = \mathcal{O}(n^{\log \log n})$ (☒) (2022)

解析:

$$\log M! = \Theta(M \log M) \Rightarrow \log([\log n]!) = \Theta(\log n \log \log n), \text{ 而 } \log n^{\log \log n} = \log \log n \log n$$

所以 $[\log n]! = \mathcal{O}(n^{\log \log n})$ 注意, 虽然答案正确, 但是以上推理有问题.

不收敛, 但有限

45

$$\text{❖ 调和级数: } h(n) = \sum_{k=1}^n \frac{1}{k} = 1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \dots + \frac{1}{n} = \ln n + \gamma + \mathcal{O}\left(\frac{1}{2n}\right) = \Theta(\log n)$$

$$\text{❖ 对数级数: } \sum_{k=1}^n \ln k = \ln \prod_{k=1}^n k = \ln n! \approx (n+0.5) \cdot \ln n - n = \Theta(n \cdot \log n)$$

$$\text{因为事实上, } n! \approx \sqrt{2\pi n} \left(\frac{n}{e}\right)^n, \ln n! \approx n \ln n - n + \frac{1}{2} \ln(2\pi n) = \Theta(n \ln n)$$

但是 $e^{n \ln n - n} \neq \Theta(e^{n \ln n})$, 也就是 $\ln f(n) = \Theta(\ln g(n))$, 不能说明 $f(n) = \Theta(g(n))$

正确的推导过程是:

$$[\log n]! = e^{\log([\log n]!)} \approx e^{\log n \log \log n - \log n} = \mathcal{O}(e^{\log n \log \log n}) = \mathcal{O}(e^{\log n^{\log \log n}}) = \mathcal{O}(n^{\log \log n}), \text{ 也就是}$$

说反过来写 $n^{\log \log n} = O([\log n]!)$ 就是错的

或者原题可能是 $[\log n]! = \Theta(n^{\log \log n})$, 这样的话就是答案就是 $\times$

6519 提出了一个更好的方法 $[\log n]! = O(n^{\log \log n}) \Leftrightarrow [\log n]! = O(\log n^{\log n}) \Leftrightarrow M! = O(M^M)$

对于级数 $\sum_{n=1}^{\infty} \frac{n!}{n^n}$, 因为 $\frac{(n+1)!}{\frac{n!}{n^n}} = \left(\frac{n}{n+1}\right)^n = \frac{1}{e} < 1$, 所以级数 $\sum_{n=1}^{\infty} \frac{n!}{n^n}$ 收敛, 所以 $\lim_{n \rightarrow \infty} \frac{n!}{n^n} = 0$

即 $M! = O(M^M)$ 成立, 所以 $[\log n]! = O(n^{\log \log n})$ 成立

10. 若某算法的运行时间满足 $T(n) = 128T\left(\frac{n}{2^{2023}}\right) + {}^{290}\sqrt{n}$, 则 $T(n) = \underline{\hspace{2cm}}$. (2023, 4 分)

解析:

根据主定理, $n^{\log_b a} = n^{\log_2 128} = n^{\frac{\log_2 128}{\log_2 2^{2023}}} = n^{\frac{7}{2023}}$, 而 $f(n) = n^{\frac{1}{290}} = n^{\frac{7}{2030}}$

所以 $T(n) = \Theta\left(n^{\frac{7}{2023}}\right)$, 新威考研整理

本章真题总结:

绪论这里主要题型有以下几种:

一是考 O, Ω, Θ 符号问题, 一般是给 $f(n) = O(g(n))$ 让我们判断对错, 要注意的地方是多重对数函数大家可能会忽略, 这里是可能会考查到的, 也就是下面图的第二行的函数, 要注意的地方其实在课上我们也强调过。对于函数增长的速度比较, 想要掌握好的话, 除了总结一些常见的增长速度的函数所在的层级, 还有就是利用两边取对数的方法, 但是要注意两边取对数是同一层级的, 那么取对数之前可能不是同一层级的。比如: $f(n) = n^3, g(n) = n^2, \lg f(n) = \Theta(\lg g(n))$

层次级别			39
常数	$O(1)$	再好不过, 但难得如此幸运	对数据结构的基本操作
	$O(\log^* n)$	在这个宇宙中, 几乎就是常数	逆Ackermann函数
对数	$O(\log n)$	与常数无限接近, 且不难遇到	有序向量的二分查找 堆、词典的查询、插入与删除
	$O(n)$	努力目标, 经常遇到	树、图的遍历
线性	$O(n \log^* n)$	几乎几乎几乎...就是线性	某些MST算法
	$O(n \log \log n)$	非常非常非常...接近线性	某些三角剖分算法
	$O(n \log n)$	最常出现, 但不见得最优	排序、EU、Huffman编码
	$O(n^2)$	所有输入对象两两组合	Dijkstra算法
平方	$O(n^2)$	不常见	矩阵乘法
立方	$O(n^3)$	P问题 = 存在多项式算法的问题	
多项式	$O(n^c)$	很多问题的平凡算法, 再尽可能优化	
指数	$O(2^n)$	绝大多数问题, 并不存在算法	
...			

二是考查主定理 (第 8 题解析中有主定理描述), 主定理一定要记住, 推导过程个人认为如果考可能就只会考第二种情况的: $f(n) = \Theta(n^{\log_b a} \cdot \log^k n)$, $T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$ 的这种情况, 而且 $f(n)$ 可能就是常数, 也就对应了 2021 年的真题。

三是考就地算法的意义.

可能会考,但是真题尚未考过的题型:

一是给一段代码,让我们分析它的时间复杂度,比如:

下列程序段的时间复杂度是:

```
int sum = 0;
for (int i = 1; i < n; i *= 2)
    for(int j = 0; j < i; j++)
        sum++;
```

(A) $O(\log n)$ (B) $O(n)$ (C) $O(n \log n)$ (D) $O(n^2)$

二是手推非主定理情况下的递归式的时间复杂度,这种题型没考过,但是本科期中期末题理考过,这种主要是通过变量替换和累次求和约掉中间项化简的方法,比如:

$$T(n) = \begin{cases} O(1) & (n \leq 1) \\ 2020 \cdot T\left(n^{\frac{1}{2020}}\right) + O(\log n) & (n \geq 2) \end{cases} \quad \text{求 } T(n) \text{ 解析式}$$

解析如下:

$$\begin{aligned} T(n) &= 2020T\left(n^{\frac{1}{2020}}\right) + O(\log n) \\ 2020T\left(n^{\frac{1}{2020}}\right) &= 2020^2T\left(n^{\frac{1}{2020^2}}\right) + O(\log n) \\ 2020^2T\left(n^{\frac{1}{2020^2}}\right) &= 2020^3T\left(n^{\frac{1}{2020^3}}\right) + O(\log n) \\ &\vdots \\ 2020^kT\left(n^{\frac{1}{2020^k}}\right) &= 2020^{k+1}T\left(n^{\frac{1}{2020^{k+1}}}\right) + O(\log n) \end{aligned}$$

$$\text{不妨设 } n = 2^{2020^k} \implies 2020^k = \log n \implies k = \log \log n \quad (1)$$

$$\text{以上各式相加, 得 } T(n) = 2020^{k+1}T(1) + (k+1)O(\log n) \quad (2)$$

$$2020^{k+1} = 2020 \log n \quad (3)$$

将 (1), (3) 和 $T(1) = O(1)$ 代入 (2) 得

$$\begin{aligned} T(n) &= 2020 \log n O(1) + (1 + \log \log n) O(\log n) \\ &= O(\log n) + O(\log n) + O(\log n \cdot \log \log n) \\ &= O(\log n \cdot \log \log n) \end{aligned}$$

三是考查递归程序的空间复杂度,要注意这里是递归深度,而不是递归实例的总数.这个知识点可以认为是和栈相结合,我把它放到这里.

总体来看绪论这部分的内容非常简单,但是考的比较多,每年必考,甚至考不止一道题.不过不太可能丢分,考生不必过于担心.

向量

1. Fibonacci 查找时以前后黄金分割点作为轴点的常系数相同 (×) (2018)

解析: 如果题意理解成分点位置是 0.618 和 0.382 的话, 分点在 0.382 的那个常系数大。Fib 查找的优化只是常系数级别的优化, 渐进时间复杂度仍然是 $\Theta(\log n)$

2. 任何情况下折半查找都比顺序查找快 (×) (2018)

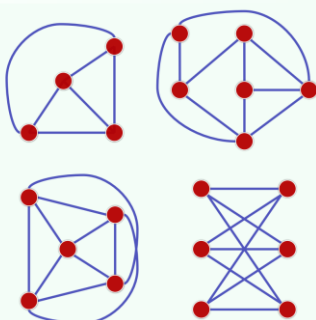
解析: 比如刚好就是第一个元素的情况下, 肯定是顺序查找更快。

3. 矩阵压缩, 一个 50×50 的对称矩阵 $a[0,49][0,49]$, 每个数据占两个数据位 (猜测是每个数据占用两个地址单元), 行优先的前提下压缩到下三角矩阵, $a[0][0]$ 起始下标对应的是十进制的 1000, $a[i][j]$ 的地址是 2000 ($i \geq j$), 则 $i = ___, j = ___ (2022)$

解析: 其实本题也可以认为是图当中的, 因为图当中有一页 PPT 提到过稀疏矩阵的压缩存储, 习题解析提到过无向图邻接矩阵的压缩存储。

缺点

- ❖ $\Theta(n^2)$ 空间, 与边数无关!
- ❖ 真会有这么多条边吗? 不妨考察一类特定的图...
- ❖ 平面图 (planar graph): 可嵌入于平面的图
- ❖ Euler's formula (1750):
 $v - e + f - c = 1$, for any PG
- ❖ 平面图: $e \leq 3 \times n - 6 = O(n) \ll n^2$
此时, 空间利用率 $\approx 1/n$
- ❖ 稀疏图 (sparse graph)
空间利用率同样很低, 可采用压缩存储技术



545

[6-4] 考查无向图的邻接矩阵表示法。

a) 试通过将二维邻接矩阵映射至一维向量, 提高空间利用率。

【解答】

无向图的邻接矩阵必然对称, 亦即, $A[i][j] = A[j][i]$ 对合法的任意 i 和 j 均成立。因此, 邻接矩阵的上或下半角完全可以不必记录, 并将剩余部分转化并压缩为一维向量 A' 。

这里, 不妨仅保存其中的下半三角区域 (含对角线), 即所有满足 $0 \leq j \leq i < n$ 的元素 $A[i][j]$ 。于是如图 x6.2 所示不难验证, 可以在这些元素与向量 A' 之间建立如下——对应关系:

$$A[i][j] \leftrightarrow A'[i(i+1)/2 + j]$$

或者等价地:

$$A'[k] \leftrightarrow A[i][k - \frac{i(i+1)}{2}], \text{ 其中 } i = \lfloor \frac{\sqrt{8k+1}-1}{2} \rfloor$$

如此所得一维向量 A' 的长度为 $n(n+1)/2$, 大致为未压缩之前的一半。但就渐进意义而言, 空间复杂度依然为 $O(n^2)$ 。

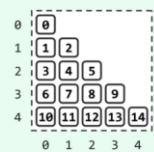


图 x6.2 将 5×5 的对称矩阵压缩至长度为 15 的一维向量

直接利用习题解析 6-4 结论:

$a'[0]$ 对应 $a[0][0]$, 对应的地址是 1000

$a'[k]$ 对应 $a[i][j]$, 对应的地址是 2000

$$k = 500, i = \left\lfloor \frac{\sqrt{8k+1}-1}{2} \right\rfloor = \left\lfloor \frac{\sqrt{4001}-1}{2} \right\rfloor = 31, \text{ 新威考研整理}$$

$$j = k - \frac{i(i+1)}{2} = 500 - \frac{31 \times (31+1)}{2} = 4.$$

4. 向量只要在即将溢出前使容量加倍, 并在装填因子不足 50% 时容量减半, 便可保证消耗在容量调整的分摊时间复杂度为常数 (×) (2023)

解析:

新威考研】所有, 仅限个人学习, 严禁任何形式的录制、传播和帐号分享。

```

if ( _capacity < DEFAULT_CAPACITY << 1 ) return; //不致收缩到DEFAULT_CAPACITY以下
T* oldElem = _elem; _elem = new T[_capacity >> 1]; //容量减半
for ( Rank i = 0; i < _size; i++ ) _elem[i] = oldElem[i]; //复制原向量内容
delete [] oldElem; //释放原空间
}

```

课上还强调过这一点↑

```

template <typename T> void Vector<T>::shrink() { //装填因子过小时压缩向量所占空间
    if ( _capacity < DEFAULT_CAPACITY << 1 ) return; //不致收缩到DEFAULT_CAPACITY以下
    if ( _size << 2 > _capacity ) return; //以25%为界
    T* oldElem = _elem; _elem = new T[_capacity >> 1]; //容量减半
    for ( Rank i = 0; i < _size; i++ ) _elem[i] = oldElem[i]; //复制原向量内容
    delete[] oldElem; //释放原空间
}

```

[vector shrink.h \(tsinghua.edu.cn\)](#)

缩容是以 25% 为界的, 如果以 50% 为界的话, 当向量刚刚扩容之后马上删除一个元素立刻就会引起缩容. 这样分摊时间复杂度将变成 $O(n)$

本章真题总结:

向量这里主要题型有以下几种:

一是 fib 查找和二分查找的特点, 这里要弄清楚二分查找版本 A, 版本 B, 版本 C 的特点, 当然考试如果考具体的版本情况的话肯定会给具体代码的。

二是特殊矩阵的压缩存储, 直接套公式, 非常简单.

三是动态向量扩容和缩容的分摊时间复杂度问题, 这也是书中第一次讲到分摊分析的问题对应的知识点.

可能会考, 但是真题尚未考过的题型:

一是直接问二分查找版本 A, 版本 B, 版本 C 的特点, 特点如下:

改进思路	168	实现	173
❖ 二分查找中左、右分支转向代价不平衡的问题, 也可直接解决, 比如... 每次迭代仅做 1 次关键码比较; 如此, 所有分支只有 2 个方向, 而不再是 3 个 ❖ 同样地, 轴点 mi 取作中点, 则查找每深入一层, 问题规模也缩减一半 - $e < x$: 则深入左侧的 $[lo, mi)$ - $x \leq e$: 则深入右侧的 $[mi, hi)$ ❖ 直到 $hi - lo = 1$ 才判断是否命中 ❖ 相对于版本 A, 最好 (坏) 情况下更坏 (好), 整体性能更趋均衡		<pre> template <typename T> static Rank binSearch(T * S, T const & e, Rank lo, Rank hi) { while (lo < hi) { //不变性: A[0, lo) <= e < A[hi, n) Rank mi = (lo + hi) >> 1; e < S[mi] ? hi = mi : lo = mi + 1; // [lo, mi) 或 (mi, hi) } //出口时, 必有 S[lo = hi] = M return lo - 1; //故, S[lo-1] = m } //留意与版本B的差异... </pre> ❖ 待查找区间宽度缩短至 0 而非 1 时, 算法才结束 // $lo == hi$ 转入右侧子向量时, 左边界取作 $mi+1$ 而非 mi // $A[mi]$ 会被遗漏? 无论成功与否, 返回的秩严格符合接口的语义约定... //如何证明?	

二是计算二分查找或者 fib 查找的平均查找长度:

查找长度

160

❖ $n = 7$ 时, 各元素对应的成功查找长度为

$\{4, 3, 5, 2, 5, 4, 6\}$

在等概率情况下, 平均成功查找长度

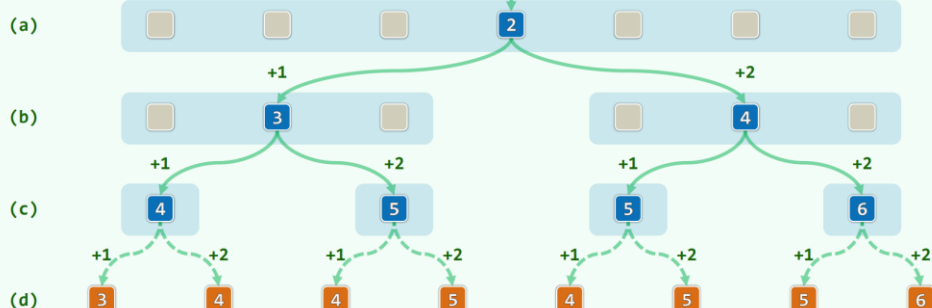
$$= 29 / 7 = 4.14$$

❖ 共 8 种失败情况, 查找长度分别为

$\{3, 4, 4, 5, 4, 5, 5, 6\}$

在等概率情况下, 平均失败查找长度

$$= 36 / 8 = 4.50$$



四是有序向量和无序向量的唯一化的

五是考查插值查找的复杂度和特点

性能

179

❖ 最坏: $hi - lo = O(n)$

// 具体实例?

❖ 平均: 每经一次比较, 待查找区间宽度由 n 缩至 $\sqrt{n}$ // [Yao76, PIA78], 习题解析[2-24]

$$n \rightarrow \sqrt{n} \rightarrow \sqrt{\sqrt{n}} \rightarrow \sqrt{\sqrt{\sqrt{n}}} \rightarrow \dots \rightarrow 2$$

$$n \rightarrow n^{1/2^1} \rightarrow n^{1/2^2} \rightarrow n^{1/2^3} \rightarrow \dots \rightarrow 2$$

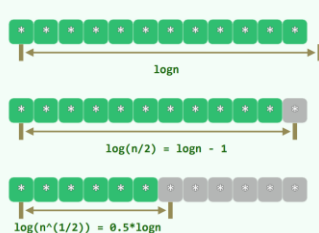
$$O(\log \log n)$$

❖ 每经一次比较,

待查找区间宽度的数值 n 开方, 有效字长 $\log n$ 减半

- 插值查找 = 在字长意义上的折半查找

- 二分查找 = 在字长意义上的顺序查找



综合评价

180

❖ 从 $O(\log n)$ 到 $O(\log \log n)$, 优势并不明显

(除非查找表极长, 或比较操作成本极高)

比如, $n = 2^{(2^5)} = 2^{32} = 4G$ 时

$$- \log_2(n) = 32$$

$$- \log_2(\log_2(n)) = 5$$

❖ 须引入乘法、除法运算

❖ 易受小扰动的干扰和“蒙骗”

❖ 实际可行的方法

- 首先通过插值查找

迅速将查找范围缩小到一定的尺度

- 然后再进行二分查找

进一步缩小范围

- 最后 (当数据项只有 200 ~ 300 时)

使用顺序查找

总体来看向量这部分的内容非常简单.不太可能丢分,考生不必过于担心.

列表

本章真题总结:

列表这里没考过题,和列表有关的排序问题放到排序中了。

可能会考,但是真题尚未考过的题型:

一是列表的插入和删除问题,要注意加链断链的顺序不要搞错了:

插入: 实现	232	删除: 实现	234
<pre> ❖ template <typename T> //前插入算法 (后插入算法完全对称) Posi(T) ListNode<T>::insertAsPred(T const & e) { //O(1) Posi(T) x = new ListNode(e, pred, this); //创建 (耗时100倍) pred->succ = x; pred = x; //次序不可颠倒 return x; //建立链接, 返回新节点的位置 } //得益于哨兵, 即便this为首节点亦不必特殊处理——此时等效于insertAsFirst(e) </pre>			
<pre> ❖ template <typename T> //删除合法位置p处节点, 返回其数值 T List<T>::remove(Posi(T) p) { //O(1) T e = p->data; //备份待删除节点数值 (设类型T可直接赋值) p->pred->succ = p->succ; p->succ->pred = p->pred; delete p; _size--; return e; //返回备份数值 } </pre>			

二是列表的游标实现 (也就是静态链表,不过不是重点):

总体来看纯列表这部分的内容非常简单,而且不是必考,没考过纯列表的问题,考了的话也不太可能丢分,考生不必过于担心。

栈和队列

1. 一组输入 MAMAMIA 入栈, 要求出栈顺序也为 MAMAMIA 共有几种方案? (2016)

- (A) 4 (B) 5 (C) 6 (D) 7

解析:

- (1) 进出进出进出进出进出进出 (2) 进进进进出出进出进出进出
(3) 进出进出进进进出出出进出进出 (4) 进出进进进出出出进出进出
(5) 进进进进进出出出出出进出进出 (6) 进进进出进出进出出出进出进出
(7) 进进进出出进进出出出进出进出

2. 五个互异节点构造的二叉搜索树有多少种? (2017)

- (A) 20 (B) 30 (C) 32 (D) 42 (E) 62

解析: $Catalan(n) = \frac{(2n)!}{n!(n+1)!}$

$Catalan(5)=42$, $Catalan(n)$ 总结如下:

情形	数量
n 个节点的二叉树的形态	$Catalan(n)$
n 个互异节点的 BST 的种类	$Catalan(n)$
n 对括号组成的合法表达式数目	$Catalan(n)$
规模为 n 个栈混洗	$Catalan(n)$
n 个叶节点的真二叉树的种类	$Catalan(n-1)$

3. 有如下逆波兰式结果为 2016, 问? 中的运算符号是多少? (2017)

0 ! * 2 2 * 6 + ^ 18 8 ? 9 / *

- (A) + (B) * (C) ^ (D) ! (E) /

解析: 2018 年也考过类似的

(1) $0!1+23!4+^{\wedge}56!7*8!/?/-9+$

(2) $11+23!4+^{\wedge}56!7*8!/?/-9+$

(3) $223!4+^{\wedge}56!7*8!/?/-9+$

(4) $2264+^{\wedge}56!7*8!/?/-9+$

(5) $2210^{\wedge}56!7*8!/?/-9+$

(6) $21024*56!7*8!/?/-9+$

(7) $204856!7*8!/?/-9+$

(8) $204857207*8!/?/-9+$

(9) $2048550408!/?/-9+$

(10) $20485504040320/?/-9+$

发现如果 $? = /$

(11) $20485\frac{1}{8}/-9+$

(12) $204840-9+$

(13) $20089+$

(14) 2017

4. 对于逆波兰式 $0!1+23!4+^{\wedge}56!7*8!/?/-9+$ 的值等于 2017, 则 $?$ 处的运算符为 () (2018)

(A) 加号 (B) 减号 (C) 乘号 (D) 除号 (E) 乘方 (F) 阶乘

解析: 原题可能不是这样, 原理和第 3 题 (2017) 一样的, 没有什么难度.

5. 非法表达式 $(12)3+!4*+5$, 执行 evaluate 算法后的结果是 () (2018)

(A) 99 (B) 89 (C) 88 (D) 98

解析: 要注意非法表达式未必都有值.

表 4.3 非法表达式“(12)3+!4*+5”的“求值”过程			
表达式	运算符栈	操作数栈	注解
(1 2) 3 + ! 4 * + 5 \$	\$		表达式起始标识入栈
((1 2) 3 + ! 4 * + 5 \$	\$ (		左括号入栈
((1 2) 3 + ! 4 * + 5 \$	\$ (	12	操作数 12 入栈
((1 2) 3 + ! 4 * + 5 \$	\$	12	左括号出栈
((1 2) 3 + ! 4 * + 5 \$	\$	12 3	操作数 3 入栈
((1 2) 3 + ! 4 * + 5 \$	\$ +	12 3	运算符 ‘+’ 入栈
((1 2) 3 + ! 4 * + 5 \$	\$ + !	12 3	运算符 ‘!’ 入栈
((1 2) 3 + ! 4 * + 5 \$	\$ + !	12 3 4	操作数 4 入栈
((1 2) 3 + ! 4 * + 5 \$	\$ +	12 3 24	运算符 ‘!’ 出栈执行
((1 2) 3 + ! 4 * + 5 \$	\$ + *	12 3 24	运算符 ‘*’ 入栈
((1 2) 3 + ! 4 * + 5 \$	\$ +	12 72	运算符 ‘*’ 出栈执行
((1 2) 3 + ! 4 * + 5 \$	\$	84	运算符 ‘+’ 出栈执行
((1 2) 3 + ! 4 * + 5 \$	\$ +	84	运算符 ‘+’ 入栈
((1 2) 3 + ! 4 * + 5 \$	\$	84 5	操作数 5 入栈
((1 2) 3 + ! 4 * + 5 \$	\$	89	运算符 ‘+’ 出栈执行
((1 2) 3 + ! 4 * + 5 \$		89	表达式起始标识出栈
((1 2) 3 + ! 4 * + 5 \$			返回唯一的元素 89

6. 有 2019 个叶节点的真二叉树数目少于 2018 对括号可以组成的合法表达式数目。 (X) (2019)

解析: 实际是等于的, 见第 2 题表格

7. 程序执行过程中, 调用栈中若同时有多帧对应同一函数, 则它们必然依次紧邻排列。 (×) (2019)

解析: 未必, 比如函数 $f(n)$ 调用 $g(n)$, $g(n)$ 调用 $f(n-1)$ 的情况, 函数 f 被函数 g 隔开了。

8. 相对于常规表达式, 逆波兰式在求值计算时为何效率更高? 前者转换为后者所需成本已相当于一次常规表达式求值, 那么转换的意义何在? (2019, 4 分)

解析: 通常同一个表达式会做多次表达式求值 (比如在一个循环中), 在多次进行同样的计算时, 不再需要比较优先级而加快运算速度, 总体来说效率更高。

9. 对于逆波兰式 $0!1+23!4+^{\wedge}\times 56!7\times 8!/?/-9+$ 的值等于 2017, 则 ? 处运算符为 () (2020)

(A) 加号 (B) 减号 (C) 乘号 (D) 除号

解析: 原题可能不是这样, 原理和第 3 题 (2017) 一样的, 没有什么难度。

10. 对于叶节点个数为 () 的真二叉树, 其数量等于 2018 对括号所组成的合法表达式数量? (2021)

(A) 2018 (B) 2019 (C) 4038 (D) 4039

解析: 和 2017 年类似, 主要是把那个对应的表格 (第 2 题, 2017) 记住就可以了。

11. 考察加 (+), 减 (-), 乘 ($\times$), 除 ($/$), 乘方 ($^{\wedge}$), 阶乘 (!) 运算 (2021, 6 分)

试将中缀表达式 “ $(0! + 1) \times 2^{\wedge}(3! + 4) - (5! / 6 + (7 - (8 - 9)))$ ” 转换为逆波兰表达式, 并按如下格式分段填写

--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

解析:

✧ 加括号: $(0! + 1) \times 2^{\wedge}(3! + 4) - (5! / 6 + (7 - (8 - 9)))$ 变成

$((((0)! + 1) \times (2^{\wedge}((3)! + 4))) - (((5)! / 6) + (7 - (8 - 9))))$

✧ 将各运算符后移, 使之紧邻于其对应的右括号的右侧:

$((((0)! 1) + (2 ((3)! 4) +)^{\wedge}) \times (((5)! 6) / (7(8 9) -) -) +) -$

✧ 去掉所有括号, 稍作整理

$0! 1 + 2 3! 4 +^{\wedge} \times 5! 6 / 7 8 9 - - + -$

✧ 最后写出答案

0	!	1	+	2	3	!	4	+	^	×	5	!	6	/	7	8	9	-	-	+	-
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

12. 如下逆波兰表达式的值若为 2023, 则被遮挡的那个运算符是 _____. (2023, 4 分)

0	!	1	2	+	3	4	+	?	+	5	!	6	7	+	8	-	9	*	+	-
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

必须填写运算符及其对应的中文 (+ 加法, - 减法, * 乘法, / 除法, ^ 乘方, ! 阶乘)

解析:

$$0!12+34+?+5!67+8-9*+-$$

$$112+34+?+5!67+8-9*+-$$

$$1334+?+5!67+8-9*+-$$

$$137?+5!67+8-9*+-, \text{不妨设 } 37? \text{ 的运算结果为 } X$$

$$1X+5!67+8-9*+-$$

$$(1+X)5!67+8-9*+-$$

$$(1+X)120,67+8-9*+-$$

$$(1+X)120,13,8-9*+-$$

$$(1+X)120,5,9*+-$$

$$(1+X)120,45,+$$

$$(1+X)165-$$

$$X-164=2023$$

$$X=37?=2187$$

$$?==^$$

本章真题总结:

栈和队列这里主要题型有以下几种:

一是考进栈顺序是 $a_1 a_2 \cdots a_n$, 输出顺序是 $a_{p_1} a_{p_2} \cdots a_{p_n} (p_i \in (1, 2, \cdots, n), i=1, 2, \cdots, n)$, 问有多少种可能性, 另一个考法是进栈顺序是 $a_1 a_2 \cdots a_n$ 存不存在输出顺序是 $a_{p_1} a_{p_2} \cdots a_{p_n} (p_i \in (1, 2, \cdots, n), i=1, 2, \cdots, n)$, 比较简单, 老老实实推就可以了。

二是考 Catalan 数, 绝对的重点, 一是要记住 Catalan(n) 的公式, 二是要记住哪些情况是 Catalan(n), 如果考了不属于表格 (第 2 题, 2017) 中的情况, 要找尝试找对应的等价关系, 找出来和表格中的已知情况的关系。

三是考逆波兰表达式求值, 已知结果问中间的某个符号, 这个 2017, 2018, 2020 都考过, 老老实实算就可以了, 送分的, 没有什么值得说的地方。

四是考逆波兰表达式特点 (2019), 值得注意的是, 题目中也说了逆波兰表达式计算的快不假, 但是从中缀表达式转化成逆波兰表达式同样会消耗时间, 而这个时间足以算出表达式的值。

五是考非法表达式求值问题,习题解析 4-12,一个小问题,知道怎么回事就能得分。

六是给定中缀表达式让转成逆波兰表达式。方法如第 9 题 (2021) 过程。

可能会考,但是真题尚未考过的题型:

一是问中缀表达式求值过程中,运算符栈和操作数栈的最大规模:

表 x4.1 表达式求值算法实例

表达式	运算符栈	操作数栈	注解
$(0!+1)*2^{(3!+4)}-(5!-67-(8+9))\$$	\$		表达式起始标识入栈
$(0!+1)*2^{(3!+4)}-(5!-67-(8+9))\$$	\$ (		左括号入栈
$(0!+1)*2^{(3!+4)}-(5!-67-(8+9))\$$	\$ (	0	操作数 0 入栈
$(0!+1)*2^{(3!+4)}-(5!-67-(8+9))\$$	\$ (!	0	运算符 '!' 入栈
$(0!+1)*2^{(3!+4)}-(5!-67-(8+9))\$$	\$ (	1	运算符 '!' 出栈执行
$(0!+1)*2^{(3!+4)}-(5!-67-(8+9))\$$	\$ (+	1	运算符 '+' 入栈
$(0!+1)*2^{(3!+4)}-(5!-67-(8+9))\$$	\$ (+	1 1	操作数 1 入栈
$(0!+1)*2^{(3!+4)}-(5!-67-(8+9))\$$	\$ (	2	运算符 '+' 出栈执行
$(0!+1)*2^{(3!+4)}-(5!-67-(8+9))\$$	\$	2	左括号出栈
$(0!+1)*2^{(3!+4)}-(5!-67-(8+9))\$$	\$ *	2	运算符 '*' 入栈
$(0!+1)*2^{(3!+4)}-(5!-67-(8+9))\$$	\$ *	2 2	操作数 2 入栈
$(0!+1)*2^{(3!+4)}-(5!-67-(8+9))\$$	\$ * ^	2 2	运算符 '^' 入栈
$(0!+1)*2^{(3!+4)}-(5!-67-(8+9))\$$	\$ * ^ (	2 2	左括号入栈
$(0!+1)*2^{(3!+4)}-(5!-67-(8+9))\$$	\$ * ^ (	2 2 3	操作数 3 入栈
$(0!+1)*2^{(3!+4)}-(5!-67-(8+9))\$$	\$ * ^ (!	2 2 3	运算符 '!' 入栈
$(0!+1)*2^{(3!+4)}-(5!-67-(8+9))\$$	\$ * ^ (	2 2 6	运算符 '!' 出栈执行
$(0!+1)*2^{(3!+4)}-(5!-67-(8+9))\$$	\$ * ^ (+	2 2 6	运算符 '+' 入栈
$(0!+1)*2^{(3!+4)}-(5!-67-(8+9))\$$	\$ * ^ (+	2 2 6 4	操作数 4 入栈

后序过程省略,也就是说模拟求值的过程,然后观察整个过程中的栈的最大规模

二是逆波兰表达式求值过程中,给出操作符栈的括号个数,问栈的最大规模,画示意图 (习题解析 4-11)

[4-11] 教材 95 页代码 4.7 中 evaluate() 算法执行过程中的某一时刻, 设操作符栈共存有 502 个括号。

a) 此时栈的规模 (含栈底的 '\0') 至多可能多大? 为什么?

b) 请示意性地画出当时栈中的内容。

习题解析很详实,不再赘述

三是考查逆波兰表达式是否一定会节省空间 (很可能考个判断) (习题解析 4-13)

[4-13] RPN 表达式无需括号即可确定运算优先级,这是否意味着其所占空间必少于常规表达式? 为什么?

【解答】

未必。实际上, 尽管 RPN 表达式可以省去括号, 但必须在相邻的操作数、操作符之间插入特定的分隔符 (通常为空格)。这种分隔符必须事先约定, 且不能用以表示操作数或操作符, 故亦称做元字符 (meta-character)。

不难看出, RPN 表达式所引入元字符的数量, 与操作数和操作符的总数相当, 故其所占空间总量未必少于原表达式。

四是队列的问题,纯的队列问题完全没考过,912 可能认为队列没什么值得考的地方? 可能考的地

方有和栈类似的比如双端队列的进队顺序出队顺序问题,队列的判空,还有两个栈模拟队列,两个队列模拟栈。

两个栈模拟队列:

入队: 将元素进栈 A。

出队: 判断栈 B 是否为空,如果为空,则将栈 A 中所有元素 pop,并 push 进栈 B,栈 B 出栈; 如果不为空,栈 B 直接出栈。

两个队列模拟栈:

入栈: 将元素进队列 A。

出栈: 判断队列 A 中元素的个数是否为 1,如果等于 1,则出队列,否则将队列 A 中的元素以此出队列并放入队列 B,直到队列 A 中的元素留下一个,然后队列 A 出队列,再把 队列 B 中的元素出队列以此放入队列 A 中。

另外二叉树的层次遍历那里考过队列的规模的问题,分类到了二叉树内容中。

总体来看栈和队列这部分的内容非常简单,不过查考的分数很多,2016~2023 一共考了 12 道题,比绪论考的还多,一定要引起重视.不过不太可能丢分,考生不必过于担心.

树和二叉树

1. 给出中序序列 {D B A E C F} 和层次序列 {A B C D E F} 能否唯一确定一颗二叉树? 能给出步骤, 不能的话请构造其中一棵 (2016)

解析: 可以构造

层次遍历确定根是 A

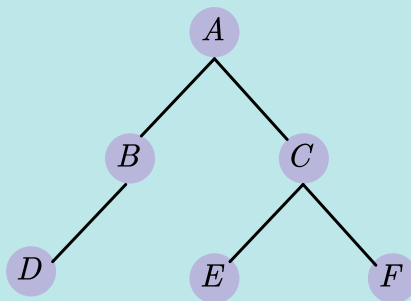
DB 在左子树, ECF 在右子树

B 一定是 A 的左孩子, C 一定是 A 的右孩子

D 的中序遍历在 B 之前, D 是 B 的左孩子

中序遍历 ECF, E 在 C 的左子树, F 在 C 的右子树

所以树可以被唯一确定, 如图



2. 二叉树遍历 (题目不完整)

给了一个递归算法, 问访问节点的顺序, 树的样子是一颗深度为 4 的二叉树。 (2016)

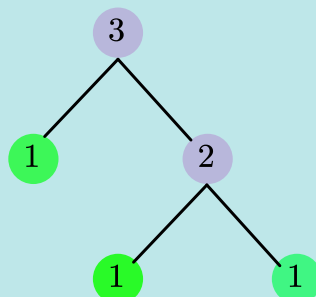
解析: 题目不完整, 找不到类似的题, 不过老老实实模拟应该没问题。

3. 哈夫曼树深度更小的节点的权值可能小于深度更大的节点的权值。 (×) (2017)

解析: 深度更小的节点的权值小于深度更大的节点的权值的话, 会和哈夫曼树的性质违背, 所以本题答案是判断为错, 有可能相同深度相同。

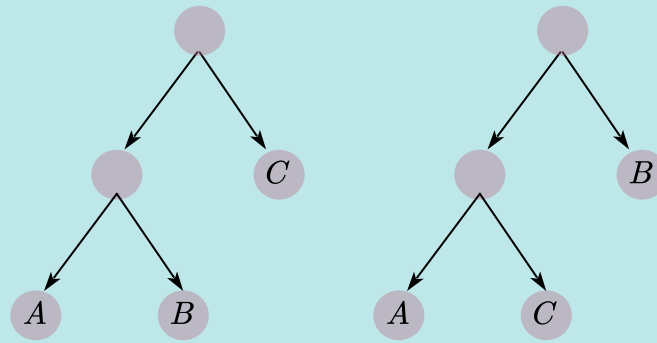
4. PFC(最优前缀编码)互换不同深度节点位置一定会破坏其性质 (×) (2018)

解析: 不一定会破坏。比如交换两个 1



5. 哈夫曼树交换不同深度子树, 编码成本必然增加。 (×) (2019)

解析: 比如 ABC 的频率都是 1/3, 其实和第 4 题没区别



6. 对于二叉树, 知道先序遍历序列和后序遍历序列不能确定层序遍历序列。 (×) (2019)

解析: 可以证明层次遍历唯一。

- [1] 对于任意的二叉树, 先序遍历中根节点在开头, 后序遍历中根节点在最后, 首先可以确定根节点。
- [2] 其次, 先序遍历的第二个节点必然是根节点的孩子节点, 后序遍历的倒数第二个节点必然是根节点的孩子节点, 如果这两个节点相同, 则说明该根节点只有一个分支。如果这两个节点不同, 则说明先序遍历的第一个节点为左孩子节点, 后序遍历的倒数第二个节点为右孩子节点, 再根据左孩子节点在左子树的后序遍历中为最后一个节点, 可以确定左右子树的先序遍历和后序遍历。
- [3] 最后, 确定了每一个节点的分支数目以及其孩子节点, 即可确定层序遍历序列, 而这个序列是唯一的。

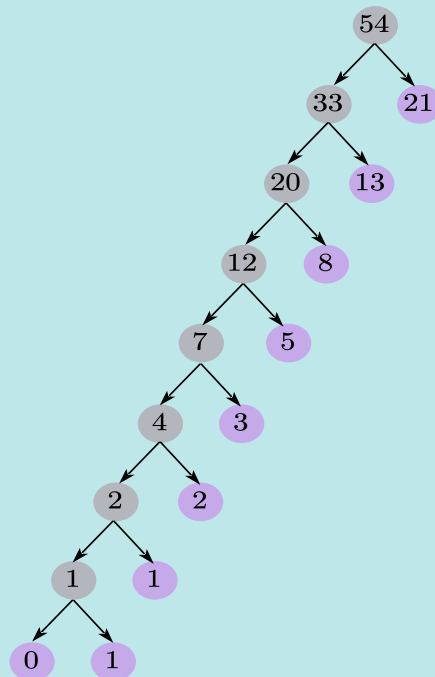
7. 对于叶节点数量为 2018 的树, 层序遍历使用的辅助队列规模绝对不超过 2018。 (√) (2019)

解析: 队列中元素最多的情况是, 叶节点都在最下层, 倒数第二层节点个数是 1009 个, 倒数第二层每一个节点出队, 都会把自己的两个孩子拉进队列, 队列中元素最多的时候是 2018 个

8. 9 个字符出现频率为 0, 1, 1, 2, 3, 5, 8, 13, 21, 其哈夫曼编码 () (2020)

- (A) 最大长度是 6 (B) 最大长度是 7
 (C) 最大长度是 8 (D) 最大长度是 9

解析: 哈夫曼树如下, 吐槽一下, 频率都是 0 了, 给它编码的意义是什么



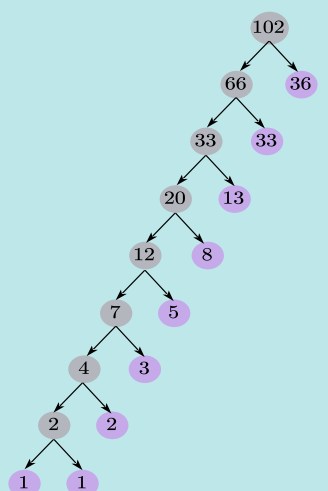
9.证明 (5 分) (2020)

给定一棵二叉树的先序和后序遍历序列,通过先序和后序遍历序列能否确定唯一层次遍历序列,若可以给出证明,不可以则说明理由

解析: 可以证明层次遍历唯一,过程见第 6 题 (2019)

10.若仅使用 9 个字符,且他们出现的频率分别为{1,1,2,3,5,8,13,33,36},则按照 Huffman 算法对长度为 2023 的随机字符串做编码,所需比特位数的期望值是____. (2023)

解析:



↑这棵树就是其中的一棵对应 Huffman 树,平均带权编码长度为

$$\frac{1 \times 8 + 1 \times 8 + 2 \times 7 + 3 \times 6 + 5 \times 5 + 8 \times 4 + 13 \times 3 + 33 \times 2 + 36 \times 1}{1 + 1 + 2 + 3 + 5 + 8 + 13 + 33 + 36} = \frac{246}{102} = \frac{41}{17}$$

故,为 2023 的随机字符串做编码需要的比特为 $2023 \times \frac{41}{17} = 4879$

本章真题总结:

树和二叉树这里主要题型有以下几种:

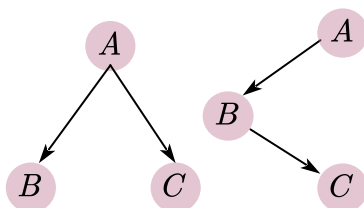
一是考二叉树的构造(给两个序列,让构造二叉树),这类题型比较简单,但是要注意先序和后序是无法确定一棵二叉树的。

二是考二叉树的遍历,给一个书上没有的遍历算法,给出遍历顺序,这种题型也不难,人工模拟算法走一遍就可以了。

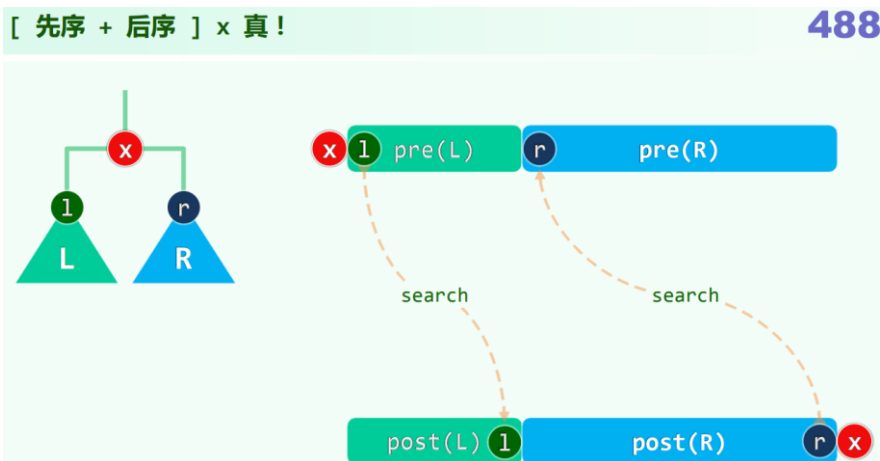
三是考哈夫曼树或者 PFC 编码树的性质,互换不同深度的节点仍然可能保持哈夫曼树的性质。

四是考给出某两个遍历顺序序列问是否能确定某遍历的顺序序列,要注意虽然先序+后序不能确定二叉树,但是可以确定二叉树的层次遍历顺序。同样层次+后序可以确定先序,但是先序+层次不能确定后序。比如

下面两棵二叉树的先序遍历和层次遍历顺序都是 ABC,但是左边二叉树的后序遍历顺序是 BCA,右边二叉树的后序遍历顺序是 CBA



另外如果加上真二叉树的条件,结论也会不一样:



先序+后序不能确定二叉树,就是因为分支的情况时候,无法确定到底是左孩子还是右孩子。

五是层次遍历辅助队列的规模问题,类似的可能会进一步地考查完全二叉树遍历问题。以及习题解析 5-18

层次遍历 480

- 考察遍历过程中的 n 步迭代...
- 前 $\lceil n/2 \rceil - 1$ 步迭代中, 均有右孩子入队
- 前 $\lceil n/2 \rceil$ 步迭代中, 都有左孩子入队
- 累计至少 $n - 1$ 次入队

辅助队列的规模 481

- 先增后减, 单峰且对称
- 最大规模 = $\lceil n/2 \rceil$
- (前 $\lceil n/2 \rceil - 1$ 次均出 1 入 2)
- 最大规模可能出现 2 次

[5-18] 考查实现如 134 页代码 5.20 所示的层次遍历算法, 设二叉树共含 n 个节点。

a) 试证明, 只要辅助队列 Q 的容量不低于 $\lceil n/2 \rceil$, 就不致于出现中途溢出的问题;

【解答】

可以证明: 在该算法执行过程中的每一步迭代之后, 若当前已经有 n 个节点入过队, 则仍在队中的至多有 $\lceil n/2 \rceil$ 个——当然, 相应地, 至少已有 $\lceil n/2 \rceil$ 个已经出队。

实际上, 对该算法稍加观察即不难发现: 每次迭代都恰有一个节点出队; 若该节点的度数为 d ($0 \leq d \leq 2$), 则随即会有 d 个节点入队。通过对已出队节点的数目做数学归纳, 即不难证明以上事实。我们将此项工作留给读者。

六是考给了叶节点的权值, 然后手工构造哈夫曼树, 白送分的, 没有可以值得讲的东西。

可能会考, 但是真题尚未考过的题型 (涉及算法题的放到最后算法题的部分):

一是树的表示, 这个比较简单, 父亲节点表示法, 孩子节点表示法, 父亲+孩子节点表示法, 长子兄弟表示法四种。

二是二叉树和有序树的转化。 (这个考过算法题了已经)

长子 + 兄弟 411

- 每个节点均设两个引用
- 纵: `firstChild()`
- 横: `nextSibling()`

- 用 $O(\deg(v) + 1)$ 时间, 可遍历 v 的所有孩子
- 再引入 `parent`, 便可在 $O(1)$ 时间内确定父亲

三是二叉树的非递归遍历, 建议看看 PPT 对应的部分。

四是增强序列问题, 可能考判断。

增强序列 489

❖ 假想地认为, 树中每个 NULL 也是“真实”节点, 并在遍历时一并输出
每次递归返回, 同时输出一个事先约定的元字符 (比如 “^”)

❖ 若将遍历序列表示为一个 `Iterator`, 则可将其定义为

```
Vector<BinNode<T>*>
```

于是在增强的遍历序列中, 这类“节点”可统一记录为 NULL

❖ 可归纳证明: 在增强的先序、中序、后序遍历序列中

- 1) 任一子树依然对应于一个子序列, 而且
- 2) 其中的 NULL 节点恰比非 NULL 节点多一个

❖ 如此, 通过对增强序列分而治之, 即可重构原树

增强序列: 实例 490

```

preorder : i d c a ^ b ^ ^ h f e ^ g ^ ^ l k j ^ ^ n m ^ p o ^ ^
inorder  : ^ a ^ b ^ c ^ d ^ e ^ f ^ g ^ h ^ i ^ j ^ k ^ l ^ m ^ n ^ o ^ p ^
postorder: ^ ^ ^ b a ^ c ^ e ^ g f ^ h d ^ j ^ k ^ m ^ o ^ p n l i
    
```

五是哈夫曼编码的平均带权编码长度计算的问题。

带权编码长度 vs. 叶节点平均带权深度 497

❖ 文件长度 $\propto$ 平均带权深度 $wald(T) = \sum_x rps(x) \times w(x)$

❖ 此时, 完全树未必就是最优编码树——比如, 考查“mamani”和“mammamia”...

总体来看, 其实纯二叉树的内容考的并不是很多, 但是有助于理解 BST, BBST, 二叉树的内容还得是学懂, 尤其是二叉树的遍历。考查的小概念题比较多, 需要抠一抠细节。

图

1. 权值都为正整数的图可以用 Dijkstra (迪佳斯特拉) 算法构造出最短路径 (✓) (2016)

解析: 可以, 基本常识

2. 程序应该是 prim 算法, 问是否能够构成最小生成树, 如果能就证明, 不能举出例子驳斥。V 表示图的点集, U 表示已经确定路径的点集, 初始时 U 为空, F 为已经确定的路径, 初始也为空。先任意取一点 u 放入 U, 然后在 V-U 中遍历 u 的邻接点, 选权值最小的边 e 和点 v 放入 F 和 U 中。(2016)

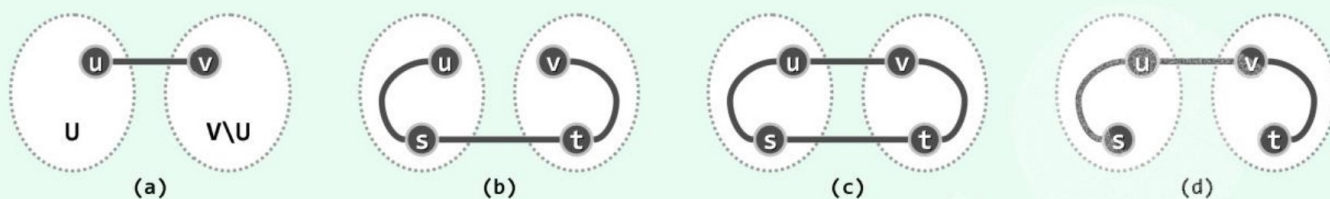
解析: 这个证明过程吧, 个人觉得把核心问题论证清楚就能得分, 不至于要求一点步骤都不差。

不妨设边的权重互异, 否则更严格的证明详见习题解析 6-26, 27, 28

图 $G=(V; E)$ 中, 顶点集 V 的任一非平凡子集 U 及其补集 $V \setminus U$ 都构成 G 的一个割(cut), 记作 $(U: V \setminus U)$ 。若边 uv 满足 $u \in U$ 且 $v \notin U$, 则称作该割的一条跨越边(Crossing edge)。因此类边连接于 U 及其补集之间, 故亦形象地称作该割的一座桥(bridge)。

Prim 算法的正确性基于以下事实: 最小支撑树总是会采用联结每一割的最短跨越边。

否则, 如图所示假设 uv 是割 $(U : V \setminus U)$ 的最短跨越边, 而最小支撑树 T 并未采用该边。于是由树的连通性, 如图(b)所示在 T 中必有至少另一跨边 st 联接该割(有可能 $s = u$ 或 $t = v$, 尽管二者不能同时成立)。同样由树的连通性, T 中必有分别联接于 u 和 s、v 和 t 之间的两条通路。由于树是极大的无环图, 故倘若将边 uv 加至 T 中, 则如图(c)所示, 必然出现穿过 u、v、t 和 s 的唯一环路。接下来, 只要再删除边 st, 则该环路必然随之消失。经过如此的一出一入, 若设 T 转换为 T' , 则 T' 依然是连通图, 且所含边数与 T 相同均为 $n-1$ 。这就意味着, T' 也是原图的一棵支撑树。就结构而言, T' 与 T 的差异仅在于边 uv 和边 st, 故二者的成本之差就是这两条边的权重之差。不难看出, 边 st 的权重必然大于身为最短跨越边的 uv, 故 T' 的总成本低于 T。这与 T 总体权重最小的前提矛盾。



3. DFS 中何时标记前向边? 何时标记后向边? (2019, 你没看错 2018 一道图的题都没考)

解析: 试图从当前顶点 v 进入处于 VISITED 状态的 u, 且 v 更早被发现。此时 (v, u) 标记前向边
试图从当前顶点 v 进入处于 DISCOVERED 状态的 u, 此时 (v, u) 标记后向边

Graph::DFS() [2/2]

574

```

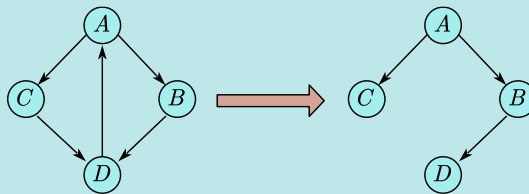
❖ for ( int u = firstNbr(v); -1 < u; u = nextNbr(v, u) ) //考察v的每一邻居u
    switch ( status(u) ) { //并视其状态分别处理
        case UNDISCOVERED: //u尚未发现, 意味着支撑树可在此拓展
            type(v, u) = TREE; parent(u) = v; DFS( u, clock ); break; //递归
        case DISCOVERED: //u已被发现但尚未访问完毕, 应属被后代指向的祖先
            type(v, u) = BACKWARD; break;
        default: //u已访问完毕 (VISITED, 有向图), 则视承袭关系分为前向边或跨边
            type(v, u) = dTime(v) < dTime(u) ? FORWARD : CROSS; break;
    } //switch

```

Data Structures & Algorithms, Tsinghua University

4.有向图经过 DFS 后有 k 条边被标记为后向边,则它不一定恰有 k 个环路。 (✓) (2020)

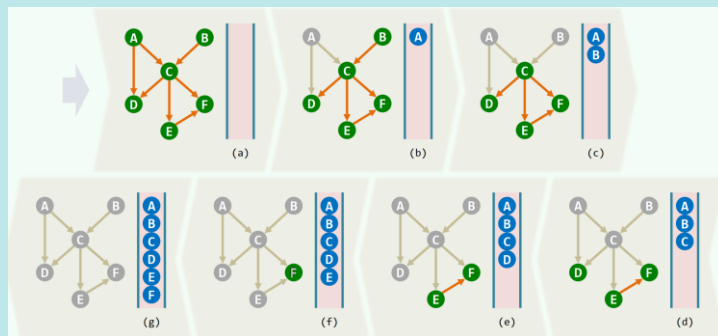
解析: 后向边数 $\leq$ 环的个数,如图



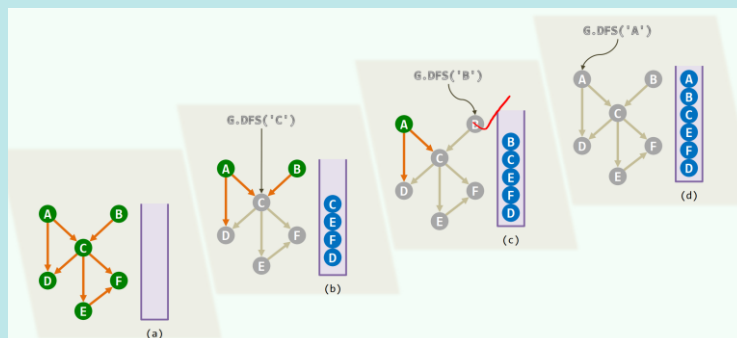
5.有向无环图的拓扑排序是 DFS 的 () (2020)

(A)被发现的顺序 (B)被发现的逆序 (C)回溯的顺序 (D)回溯的逆序

解析: 拓扑排序的零入度算法:



零出度算法:



6.深度优先搜索+拓扑排序 (10 分) (2021)

对任一有向无环图做 DFS,所有节点按照回溯的次序逆序排列,是否总是一个拓扑排序? 如果是,试予

证明: 否则, 试给处一个反例

解析: 是拓扑排序, 这个就是零出度算法

6.8.3 算法

不妨将关注点, 转至与极大顶点相对称的极小顶点。

同理, 有限偏序集中也必然存在极小元素 (同样, 未必唯一)。该元素作为顶点, 出度必然为零——比如图 6.10(b) 中的顶点 D 和 F。而在对有向无环图的 DFS 搜索中, 首先因访问完成而转换至 VISITED 状态的顶点 m, 也必然具有这一性质: 反之亦然。

进一步地, 根据 DFS 搜索的特性, 顶点 m (及其关联边) 对此后的搜索过程将不起任何作用。于是, 下一转换至 VISITED 状态的顶点可等效地理解为是, 从图中剔除顶点 m (及其关联边) 之后的出度为零者——在拓扑排序中, 该顶点应为顶点 m 的前驱。由此可见, DFS 搜索过程中各顶点被标记为 VISITED 的次序, 恰好 (按逆序) 给出了原图的一个拓扑排序。

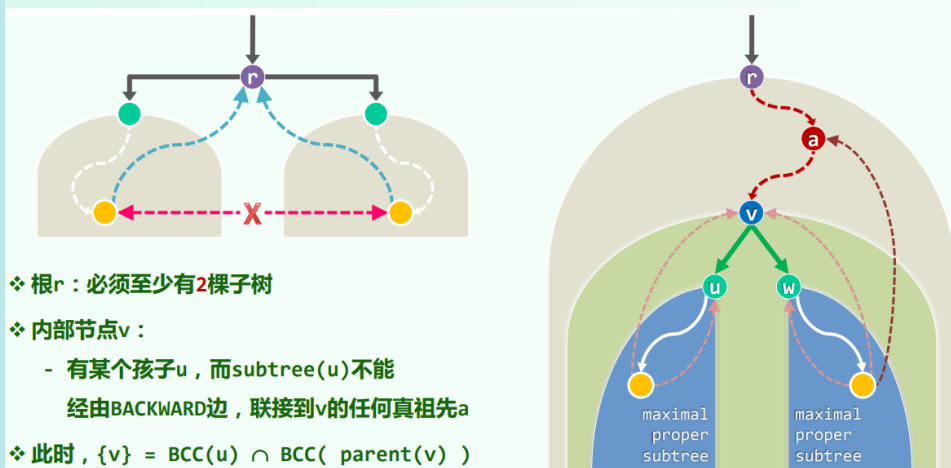
此外, DFS 搜索善于检测环路的特性, 恰好可以用来判别输入是否为有向无环图。具体地, 搜索过程中一旦发现后向边, 即可终止算法并报告 “因非 DAG 而无法拓扑排序”。

7. 对无向图某一顶点 s 进行 DFS, 若 DFS 树中 s 度数为 1, 则 s 必不为关节点 (✓) (2022)

解析: 题目说的是从某一顶点 s 进行 DFS, 从这个点进行 DFS, 我理解的话这个 s 就是 DFS 树的根了, 那么至少有两个分支, 才可能是关节点。

非叶节点

657



8. 对于无向图, 从任意顶点出发, 进行 BFS, 队列中的节点与起始节点距离的差不大于 1 (✓) (2022)

解析: 考查 BFS 细节, 但是题目回忆版描述模糊, 黄色高亮部分是我加进去的。

最短路径

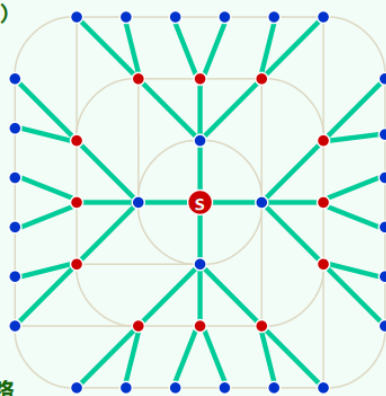
568

❖ 无向图中, 顶点 v 到 u 的 (最近) 距离记作 $\text{dist}(v, u)$

❖ BFS 过程中, 队列 Q 犹如一条贪吃蛇

- 其中的顶点按 $\text{dist}(s)$ 单调排列
- 相邻顶点的 $\text{dist}(s)$ 相差不超过 1
- 首、末顶点的 $\text{dist}(s)$ 相差不超过 1
- 由树边联接的顶点, $\text{dist}(s)$ 恰好相差 1
- 由跨边联接的顶点, $\text{dist}(s)$ 至多相差 1

❖ BFS 树中从 s 到 v 的路径, 即是二者在原图中的最短通路



9. 相对于 kruskal 算法, Prim 算法更适宜于稀疏图 (×) (2023)

解析:

初步认为指的就是没改进的 Prim 算法, 更适用于稠密图, 前两张图出自邓书, 后两张图出自李春葆书。

■ 复杂度

不难看出, 以上顶点优先级更新器只需常数的运行时间, 故由 6.10.3 节对优先级搜索算法性能的分析结论, 以上 Prim 算法的时间复杂度为 $O(n^2)$ 。作为 PFS 搜索的特例, Prim 算法的效率也可借助优先级队列进一步提高 (习题 [10-16] 和 [10-17])。

[10-16] 若能注意到教材 6.11.5 节 Prim 算法中定义的“优先级数”恰好对应于优先级队列中元素的优先级, 即可利用本章介绍的优先级队列, 改进如教材 177 页代码 6.8 所示的 $O(n^2)$ 版本。

具体地, 可首先花费 $O(n)$ 时间, 将起点 s 与其余顶点之间的 $n - 1$ 条边组织为一个优先级队列 H 。此后的每一步迭代中, 只需 $O(\log n)$ 时间即可从 H 中取出优先级数最小的边 (最短桥), 并将对应的顶点转入最小支撑树中。不过, 随后为了高效地对 H 中与刚转出顶点相关联的每一条边做松弛优化, 需要增加一个 $\text{decrease}(e)$ 接口, 在边 e 的优先级数减少后将 H 重新调整成一个堆。

a) 参照如代码 10.7 所示的 $\text{percolateUp}()$ 上滤算法为堆结构增加 $\text{decrease}()$ 接口, 要求运行时间不超过 $O(\log n)$;

【解答】

这类 $\text{decrease}()$ 接口的工作原理, 可参见习题 [10-12]。

请读者根据以上介绍和提示, 独立完成编码和调试任务。

b) 试证明, 如此改进之后 Prim 算法的效率为 $O((n + e)\log n)$, 非常适用于稀疏图;

【解答】

按照该算法, 取出每个顶点需要 $O(\log n)$ 时间, 累计 $O(n\log n)$ 时间。

所有顶点的所有邻接顶点的松弛, 在最坏情况下累计需要 $O(e\log n)$ 时间。

两项合计, 即得题中结论。

Prim() 算法中有两重 for 循环, 所以时间复杂度为 $O(n^2)$, 其中 n 为图的顶点个数。大家可以看出, Prim() 算法的执行时间与图中的边数 e 无关, 所以它特别适合用稠密图求最小生成树。

如果给定的带权连通图 G 有 n 个顶点、 e 条边, 上述改进的 Kruskal 算法中不考虑生成边数组 E 的过程, 堆排序的时间复杂度为 $O(e \log_2 e)$ 。while 循环是在 e 条边中选取 $(n-1)$ 条边, 其中的 UNION() 的执行时间为 $O(\log_2 n)$, 因此 while 循环的时间复杂度为 $O(e \log_2 n)$ 。对于连通无向图, $e \geq n-1$, 那么改进的 Kruskal 算法构造最小生成树的时间复杂度为 $O(e \log_2 e)$ 。可以看出, Kruskal 算法的执行时间仅与图中的边数有关, 与顶点数无关, 所以它特别适合用稀疏图求最小生成树。

289

10. 只要存在负权边, 则 Dijkstra 算法即使能够终止, 所输出的也不是最短路径. (✓) (2023)

解析:

消除歧义

638

❖ 各边权重均为正, 否则

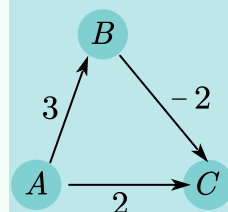
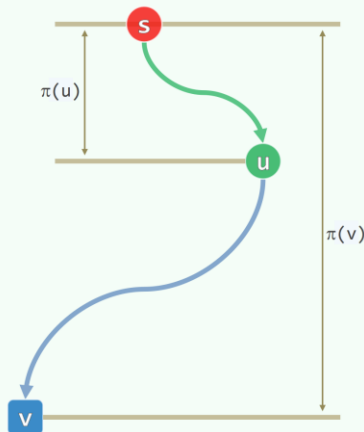
- 有可能出现总权重非正的环路
- 以致最短路径无从定义

❖ 有负权重的边时, 即便所有环路总权重皆为正

以下将介绍的 Dijkstra 算法依然可能失效

❖ 任意两点之间, 最短路径唯一

- 在不影响计算结果的前提下
- 总可通过适当扰动予以保证 (习题 [6-25])



本章真题总结:

图这里主要题型有以下几种:

一是考二 Dijkstra 算法, 相当于在问 Dijkstra 算法是干啥的, Dijkstra 算法负权边的问题, 送分题

二是考 prim 算法, 正确性的证明, 比较有难度和 prim 算法适用于稠密图的问题。

三是考 DFS 的细节, 什么时候标记的后向边? 还考了后向边的条数和环的个数的关系

四是考拓扑排序的零出度算法。

五是考了双连通分量中的关节点问题。

六是考了 BFS 的细节问题, 队列中的顶点相关问题。

可能会考, 但是真题尚未考过的题型 (涉及算法题的放到最后算法题的部分):

一是图的储存结构 (邻接矩阵, 邻接表, 关联矩阵, 邻接多重表) 的特点

邻接矩阵 + 关联矩阵

528

❖ adjacency matrix: 记录顶点之间的邻接关系

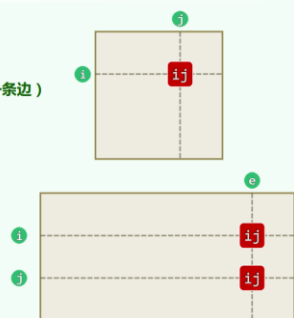
- 对应: 矩阵元素 $\Leftrightarrow$ 图中可能存在的边
- $A(i, j) = 1$ (若顶点 i 与 j 之间存在一条边)
 - $= 0$ (否则)

既然只考察简单图, 对角线统一设置为 0
空间复杂度为 $O(n^2)$, 与图中实际的边数无关

❖ incidence matrix: 记录顶点与边之间的关联关系

空间复杂度为 $O(n \cdot e) = O(n^3)$ 空间利用率 = $2e/ne = 2/n$

解决某些问题时十分有效



优点

544

❖ 直观, 易于理解和实现

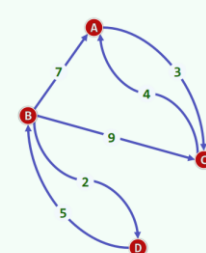
❖ 适用范围广泛

尤其适用于稠密图 (dense graph)

❖ 判断两点之间是否存在边: $O(1)$ ❖ 获取顶点的 (出/入) 度数: $O(1)$ 添加、删除边后更新度数: $O(1)$

❖ 扩展性 (scalability): 得益于 Vector 良好的控制策略, 空间溢出等情况可被“透明地”处理

	A	B	C	D
A			3	
B	7		9	2
C	4			
D		5		



缺点

545

❖ $O(n^2)$ 空间, 与边数无关!

❖ 真会有这么多条边吗? 不妨考察一类特定的图...

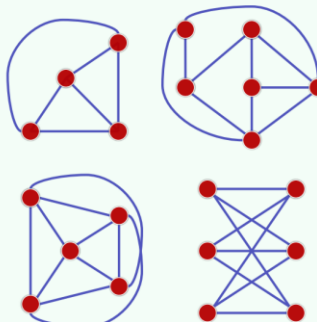
❖ 平面图 (planar graph): 可嵌入于平面的图

❖ Euler's formula (1750):

 $v - e + f - c = 1$, for any PG❖ 平面图: $e \leq 3 \times n - 6 = O(n) \ll n^2$ 此时, 空间利用率 $\approx 1/n$

❖ 稀疏图 (sparse graph)

空间利用率同样很低, 可采用压缩存储技术



空间复杂度

549

❖ 有向图 = $O(n + e)$ ❖ 无向图 = $O(n + 2e) = O(n + e)$

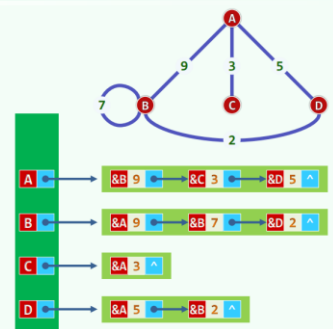
- 注意: 无向弧被重复存储

- 问题: 如何改进?

❖ 适用于稀疏图

❖ 平面图 = $O(n + 3 \times n) = O(n)$

较之邻接矩阵, 有极大改进



取舍原则

552

❖ 空间/速度

❖ 顶点类型

- bit
- int
- float
- struct
- class
- ...

❖ 弧类型 (方向 / 权值)

❖ 图类型 (稀疏 / 稠密)

	邻接矩阵	邻接表
适用场合	经常检测边的存在 经常做边的插入/删除 图的规模固定 稠密图	经常计算顶点的度数 顶点数目不确定 经常做遍历 稀疏图

二是 BFS, DFS 细节, 比如问遍历顺序有多少种可能? 让填写 DFS 的 dtime 和 ftime, 还有 DFS 中引申出来的括号引理, 以及 DFS 和 BFS 适用的情况。

括号引理

593

❖ 活跃期: $\text{active}[u] = (\text{dTime}[u], \text{fTime}[u])$

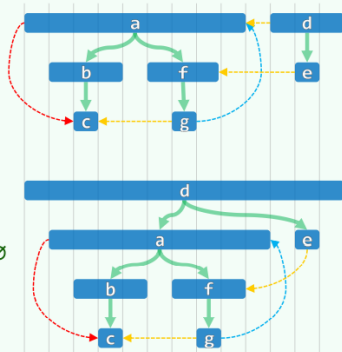
❖ 【Parenthesis Lemma】

给定有向图 $G = (V, E)$ 及其任一 DFS 森林, 则

- u 是 v 的后代 iff $\text{active}[u] \subseteq \text{active}[v]$
- u 是 v 的祖先 iff $\text{active}[u] \supseteq \text{active}[v]$
- u 与 v “无关” iff $\text{active}[u] \cap \text{active}[v] = \emptyset$

❖ 仅凭 $\text{status}[]$, $\text{dTime}[]$ 和 $\text{fTime}[]$

即可对各边分类...



连通图的支撑树 (DFS/BFS Tree)

DFS/BFS

非连通图的支撑森林

DFS/BFS

连通性检测

DFS/BFS

无向图环路检测/二部图判定

DFS/BFS

有向图环路检测

DFS

顶点之间可达性检测/路径求解

DFS/BFS

顶点之间的最短距离

BFS

直径/半径/围长/中心

BFS

欧拉回路

DFS

拓扑排序

DFS

双连通分量、强连通分量分解

DFS

...

...

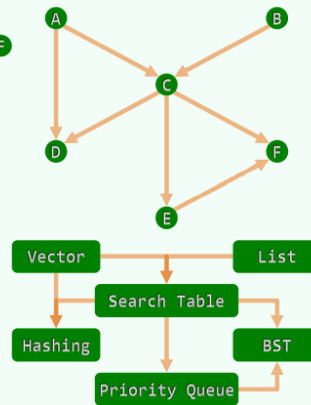
三是继续考拓扑排序, 但是问具体的拓扑排序顺序, 或者拓扑排序的零入度算法 (第 5 题的解析中有写零入度算法), 还有拓扑排序和 DAG 的关系。

偏序 ~ 极值

599



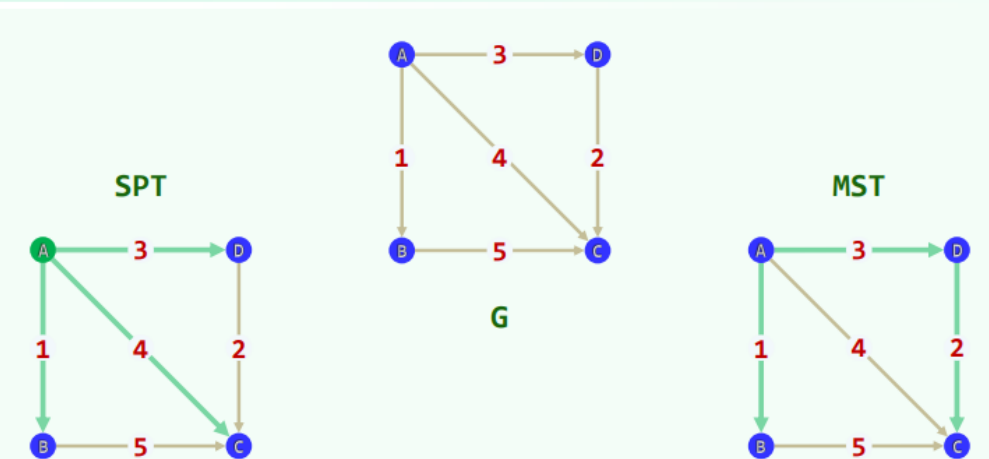
- ❖ 每个 DAG 对应于一个偏序集；拓扑排序对应于一个全序集
所谓的拓扑排序，即构造一个与指定偏序集相容的全序集
- ❖ 可以拓扑排序的有向图，必定无环 // 反之...
- ❖ 任何 DAG，都存在（至少）一种拓扑排序？是的！
- ❖ 有限偏序集必有极值元素...
- ❖ 可归纳证明，并直接导出一个算法...



四是最小生成树,最短路径树的细节问题,主要是两大算法 Prim 和 Dijkstra,另外要注意 SPT 不一定等于 MST

SPT ≠ MST

640



五是双连通分量的细节,虽然 2022 考过可能还会考,可能是判断题

六是 Kruskal 最小生成树算法（引申出并查集,在我的课上也都讲过）和 Floyd 最短路径算法

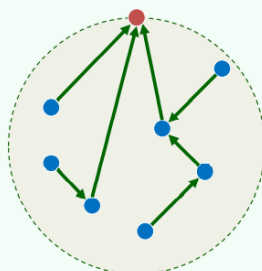
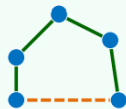
贪心策略

671

从 Dijkstra 到 Floyd-Warshall

683

- ❖ 回顾 Prim 算法
 - 最短边，迟早会被采用
 - 次短边，亦是如此
 - 再次短者，则未必 // 回路！
- ❖ Kruskal：贪心原则
 - 根据代价，从小到大依次尝试各边
 - 只要“安全”，就加入该边
- ❖ 但是，每步局部最优 = 全局最优？
- ❖ 确实，Kruskal 很幸运...



- ❖ 给定带权网络 G，计算其中所有点对之间的最短距离
- ❖ 应用：确定 G 的中心点（center）
 - $\text{radius}(G, s) = s$ 的半径 = 所有顶点到 s 的最大距离
 - 中心点 = 半径最小的顶点 s
- ❖ 直觉：依次将各顶点作为源点，调用 Dijkstra 算法
时间 = $n \times O(n^2) = O(n^3)$ —— 可否更快？
- ❖ 思路：图矩阵 ~ 最短路径矩阵
- ❖ 效率： $O(n^3)$ ，与执行 n 次 Dijkstra 相同 —— 既然如此，为何还要用 F.W.？
- ❖ 优点：形式简单、算法紧凑、便于实现；允许负权边（尽管仍不能有负权环路）

搜索树

1. 二叉搜索树中最大的节点 () (2016)

(A) 仅有左孩子, 没有右孩子

(B) 仅有右孩子, 没有左孩子

(C) 既有左孩子, 又有右孩子

(D) 没有左孩子, 也没有右孩子

解析: 回忆版有问题, 只能确定没有右孩子, 比如只有一个根节点的 BST, 那根节点就是最大节点.

2. 在 AVL 树中, () 可能会发生两次旋转调整? (2016)

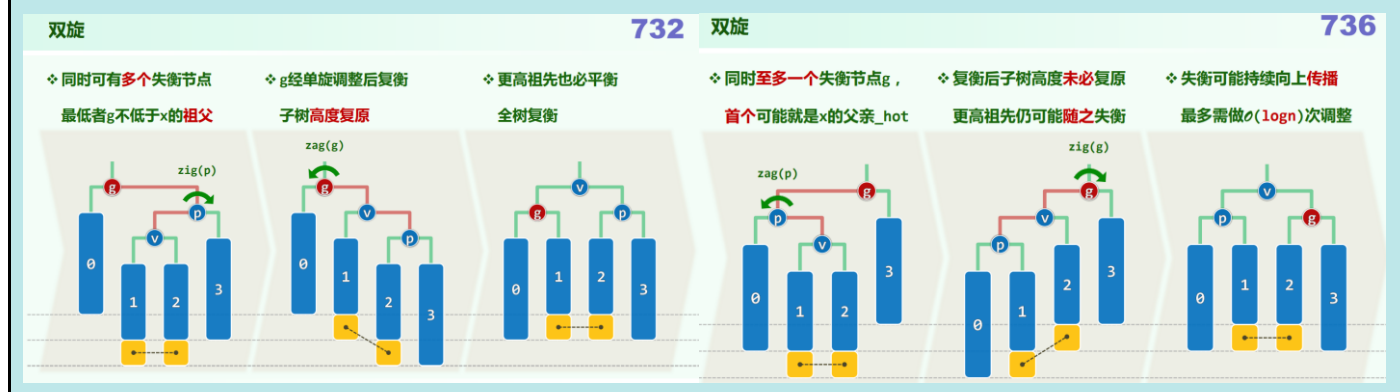
(A) 添加、删除节点操作

(B) 仅删除节点操作

(C) 仅添加节点操作

(D) 添加、删除节点都

解析: 插入和删除都有可能



3. 将关键字 1, 2, 3, ..., 2016 插入初始为空的平衡二叉树中, 假设只有一个根节点的二叉树高度为 0, 那么最终二叉树的高度是多少? (2017)

(A) 8 (B) 9 (C) 10 (D) 11 (E) 12

解析: 1023 个节点时, 二叉树刚好是满二叉树, 高度是 9, 2016 时候高度是 10。所以选择 C
其实隐含着考查了一个习题解析 7-20 的结论。

[7-20] 试证明, 按递增次序将 $2^{h+1} - 1$ 个关键字插入初始为空的 AVL 树中, 必然得到高度为 h 的满树。

【解答】

首先, 考察 AVL 的右侧分支。对照 AVL 树的重平衡算法不难发现, 在这样的插入过程中, 该分支上沿途上各节点 v 始终满足以下不变性:

1) v 的左子树必为满树;

2) $\text{height}(\text{rc}(v)) - 1 \leq \text{height}(\text{lc}(v)) \leq \text{height}(\text{rc}(v))$

实际上, 在这一系列的插入操作过程中出现的每一次失衡, 都可以通过 zag 单旋予以修复。如教材 196 页图 7.15(a) 所示, 若 T_0 、 T_1 和 T_2 都是满树, 则旋转之后应如图(b)所示, 节点 g 与 T_0 和 T_1 必然也构成一棵 (增高一层的) 满树。

4. 对于同一个长度为 n 的序列分别按照递增和递减的顺序构造 AVL 树, 那么“存在正整数 k , 使 $n = 2^k - 1$ ”是“两次构造的 AVL 树相同”的 () (2018)

(A)充分不必要条件 (B)必要不充分条件 (C)充分必要条件 (D)既不充分也不必要条件

解析: “存在正整数 k 使 $n = 2^k - 1$ ”是“两次构造的 AVL 相同”的充要条件大前提是按照递增和递减的顺序构造两棵 AVL 树,那么 $n = 2^k - 1$ 是这两棵树相同的什么条件.

注意到递增和递减顺序构造的两棵树“形态”上是镜像的,就是一个树左右翻转一下就是另一棵树的形态.因为递增和递减顺序操作是反过来的,递增的时候作为右孩子插入进去,递减的时候作为左孩子插入进去,对应的旋转也是反方向的.

充分性: $n = 2^k - 1$,这两棵树都是满二叉树,必然相同

必要性: 两棵树相同,又因为是镜像的,所以必然是满二叉树的形态,不可能出现对称但不满的情况的.那样一定不是单调插入的.

5. AVL 树在插入一个节点后可能引起 $O(\lg n)$ 次局部重构. (X) (2020)解析: 插入是 $O(1)$,删除是 $O(\lg n)$,看 ppt 的红色的部分.

实现

733 实现

737

```

template <typename T> BinNodePosi(T) AVL<T>::insert( const T & e ) {
    BinNodePosi(T) & x = search( e ); if ( x ) return x; //若目标尚不存在
    BinNodePosi(T) xx = x = new BinNode<T>( e, _hot ); _size++; //则创建新节点
    // 此时,若x的父亲_hot增高,则祖父有可能失衡.故以下从_hot起,向上逐层检查各代祖先
    for ( BinNodePosi(T) g = _hot; g; g = g->parent )
        if ( !AvlBalanced( *g ) ) { //一旦发现g失衡,则通过调整恢复平衡
            FromParentTo( *g ) = rotateAt( tallerChild( tallerChild( g ) ) );
            break; //g复衡后,局部子树高度必然复原;其祖先亦必如此,故调整结束
        } else //否则(在依然平衡的祖先处),只需简单地
            updateHeight( g ); //更新其高度(平衡性虽不变,高度却可能改变)
    return xx; //返回新节点:至多只需一次调整
}

```

```

template <typename T> bool AVL<T>::remove( const T & e ) {
    BinNodePosi(T) & x = search( e ); if ( !x ) return false; //若目标尚不存在
    removeAt( x, _hot ); _size--; //则在按BST规则删除之后,_hot及祖先均有可能失衡
    // 以下,从_hot出发逐层向上,依次检查各代祖先
    for ( BinNodePosi(T) g = _hot; g; g = g->parent ) {
        if ( !AvlBalanced( *g ) ) //一旦发现g失衡,则通过调整恢复平衡
            g = FromParentTo( *g ) = rotateAt( tallerChild( tallerChild( g ) ) );
        updateHeight( g ); //并更新其高度
    } //可能需做过O(logn)次调整;无论是否做过调整,全树高度均可能下降
    return true; //删除成功
}

```

6. AVL 树 (4+2+8=14 分) (2021)

我们知道,AVL 树在删除一个节点后如果失衡,可以通过适当的调整来恢复平衡

(1)为确定应在何处,以何种方式完成这样的调整我们需要在某相邻的三层中确定三个节点,并按高度递减次序分别命名为 g, p 和 v ,试说明在算法中是如何找到它们的?

解析:

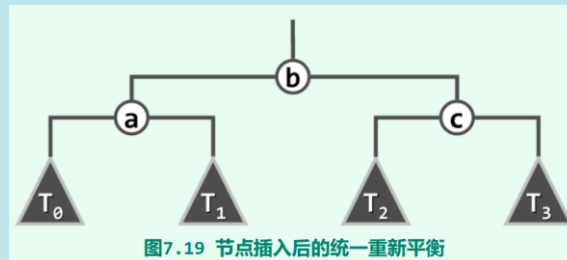
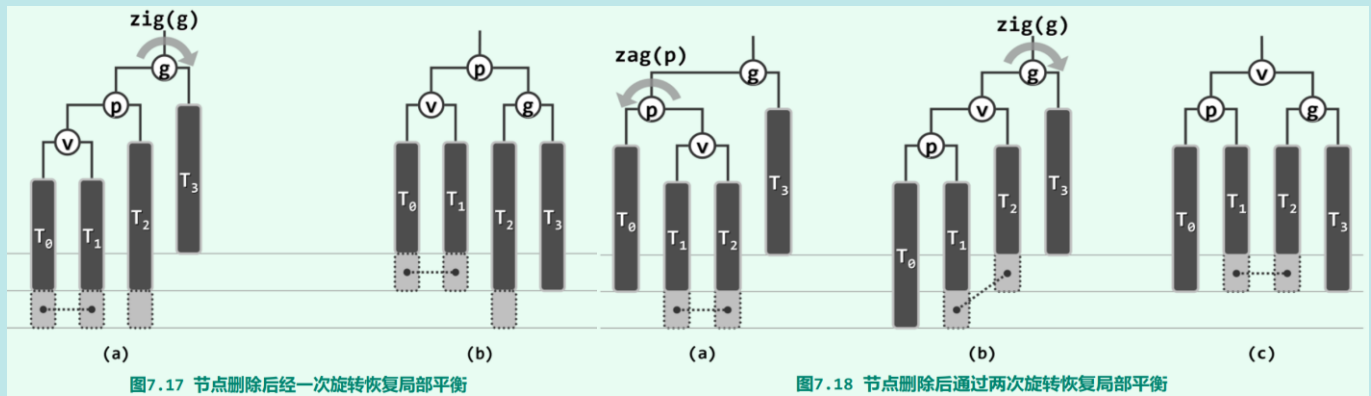
■ 重平衡

与插入操作同理,从 $_hot$ 节点 (7.2.6 节) 出发沿 $parent$ 指针上行,经过 $O(\lg n)$ 时间即可确定 $g(x)$ 位置。作为失衡节点的 $g(x)$,在不包含 x 的一侧,必有一个非空孩子 p ,且 p 的高度至少为 1。于是,可按以下规则从 p 的两个孩子 (其一可能为空) 中选出节点 v : 若两个孩子不等高,则 v 取作其中的更高者;否则,优先取 v 与 p 同向者 (亦即, v 与 p 同为左孩子,或者同为右孩子)。

以下不妨假定失衡后 $g(x)$ 的平衡因子为 +2 (为 -2 的情况完全对称)。根据祖孙三代节点 $g(x)$ 、 p 和 v 的位置关系,通过以 $g(x)$ 和 p 为轴的适当旋转,同样可以使得这一局部恢复平衡。

(2)如果将各种调整的方式统一为“(3+4) - 重构”,那么在调用 $connect34(a,b,c,T_0,T_1,T_2,T_3)$ 时,七个参数共有多少种可能的情况?

解析: 四种情况, 另外两种情况其实就是图 7.17 和图 7.18 的镜像。



(3) 试按如下表格形式, 逐行地具体列出这些情况 (次序无所谓)

可能的情况	a	b	c	T0	T1	T2	T3
#1	v	p	g	v->lc	v->rc	p->rc	g->rc
#2	p	v	g	p->lc	v->lc	v->rc	g->rc
#3	g	p	v	g->lc	p->lc	v->lc	v->rc
#4	g	v	p	g->lc	v->lc	v->rc	p->rc

解析: 见代码 7-14

```

/*****
 * BST节点旋转变换统一算法 (3节点 + 4子树), 返回调整之后局部子树根节点的位置
 * 注意: 尽管子树根会正确指向上层节点 (如果存在), 但反向的连接须由上层函数完成
 *****/
template <typename T> BinNodePosi(T) BST<T>::rotateAt ( BinNodePosi(T) v ) { //v 为非空孙辈节点
    BinNodePosi(T) p = v->parent; BinNodePosi(T) g = p->parent; //视v、p和g相对位置分四种情况
    if ( IsLChild ( *p ) ) /* zig */
        if ( IsLChild ( *v ) ) { /* zig-zig */
            p->parent = g->parent; //向上联接
            return connect34 ( v, p, g, v->lc, v->rc, p->rc, g->rc );
        } else { /* zig-zag */
            v->parent = g->parent; //向上联接
            return connect34 ( p, v, g, p->lc, v->lc, v->rc, g->rc );
        }
    else /* zag */
        if ( IsRChild ( *v ) ) { /* zag-zag */
            p->parent = g->parent; //向上联接
            return connect34 ( g, p, v, g->lc, p->lc, v->lc, v->rc );
        } else { /* zag-zig */
            v->parent = g->parent; //向上联接
            return connect34 ( g, v, p, g->lc, v->lc, v->rc, p->rc );
        }
}

```

代码 7.14 AVL 树的统一重平衡

7. 由关键码 {0, 1, 2, ..., 10} 组成, 且所有节点都是偶数度的二叉搜索树, 共有 42 棵 (2022)

解析: $Catalan(5) = 42$, 如果一下子没想起来这个是卡特兰数, 也可以用以下的方法. 本题也可以放在栈那一章中.

也有另外一种解法:

$T(n)$ 代表 n 个节点的偶数度的 BST 的棵数

$$T(11) = 2T(9)T(1) + 2T(7)T(3) + T(5)T(5)$$

$$T(9) = 2T(7)T(1) + 2T(5)T(3)$$

$$T(7) = 2T(5)T(1) + T(3)T(3)$$

$$T(5) = 2T(3)T(1)$$

$$T(3) = T(1)T(1)$$

$$T(1) = 1, \text{ 因此}$$

$$T(3) = T(1)T(1) = 1$$

$$T(5) = 2T(3)T(1) = 2$$

$$T(7) = 2T(5)T(1) + T(3)T(3) = 5$$

$$T(9) = 2T(7)T(1) + 2T(5)T(3) = 14$$

$$T(11) = 2T(9)T(1) + 2T(7)T(3) + T(5)T(5) = 42$$

8. AVL 树删除 x, g 失衡后, 经过调整, 其高度可能不能保持原高度 (✓) (2022) (x 和 g 是我加上去的,

网传回忆版并没有 x 和 g 这两个字母, 感觉不通顺, 故加入)

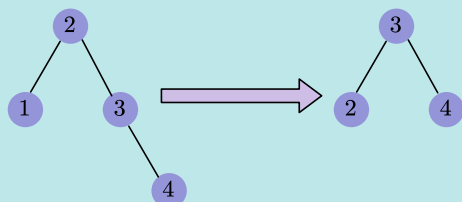
解析: 注意即便 g 未失衡, 高度亦可能降低

以上算法过程, 可描述并实现如代码 7.12 所示。

```
1 template <typename T> bool AVL<T>::remove ( const T& e ) { //从AVL树中删除关键码e
2     BinNodePosi(T) & x = search ( e ); if ( !x ) return false; //确认目标存在 (留意_hot的设置)
3     removeAt ( x, _hot ); _size--; //先按BST规则删除之 (此后, 原节点之父_hot及其祖先均可能失衡)
4     for ( BinNodePosi(T) g = _hot; g; g = g->parent ) { //从_hot出发向上, 逐层检查各代祖先g
5         if ( !AvlBalanced ( *g ) ) //一旦发现g失衡, 则 (采用 "3 + 4" 算法) 使之复衡, 并将该子树联至
6             g = FromParentTo ( *g ) = rotateAt ( tallerChild ( tallerChild ( g ) ) ); //原父亲
7         updateHeight ( g ); //并更新其高度 (注意: 即便g未失衡, 高度亦可能降低)
8     } //可能需做Omega(logn)次调整——无论是否做过调整, 全树高度均可能降低
9     return true; //删除成功
0 } //若目标节点存在且被删除, 返回true; 否则返回false
```

代码 7.12 AVL 树节点的删除

g 降低的高度和对应的情况, 有小伙伴总结过吗?



9.按照二叉搜索树规则在 AVL 树中插入一个节点后,若造成失衡了,且在定位祖孙三代节点 g 、 p 、 v 后,只需要围绕 g 做一次旋转即可使局部恢复平衡了,则在旋转之前节点 p 的平衡因子不可能是零 (✓) (2023)

解析:

单旋

731

✧ 同时可有多个失衡节点

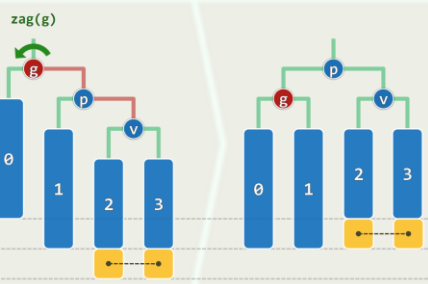
✧ g 经单旋调整后复衡

✧ 更高祖先也必平衡

最低者 g 不低于 x 的祖父

子树高度复原

全树复衡



单旋的情况, p 的平衡因子一定是 1 或者 -1

本章真题总结:

搜索树这里主要题型有以下几种:

一是考 BST 的中序遍历单调非降的结论 (最大的节点无右孩子), 送分题。

二是考节点互异的真二叉 BST 的种类, 就是还是考卡特兰数。

三是考 AVL 的插入或删除的局部重构的次数

四是考 AVL 单调插入的结论

五是考 AVL 插入, 删除和 “3+4 调整” 的细节, g 的高度, p 的平衡因子等问题。

可能会考, 但是真题尚未考过的题型 (涉及算法题的放到最后算法题的部分):

一是 BST 的查找, 插入, 删除的过程, 这个也比较简单

二是 BST 的平均树高, 要分成随机生成和随机组成两种情况。

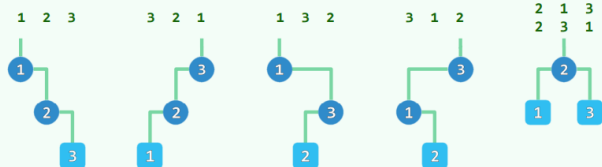
随机生成

712

随机组成

713

✧ 考查 n 个互异词条 $\{e_1, e_2, e_3, \dots, e_n\}$, 对任一排列 $\sigma = (e_{i_1}, e_{i_2}, e_{i_3}, \dots, e_{i_n}) \dots$

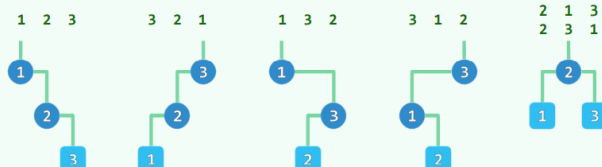


✧ 从空树开始, 反复调用 $\text{insert}()$ 接口将各词条依次插入, 得到 $T(\sigma)$

✧ 若假定任一排列 σ 作为输入的概率均等 $1/n!$

则由 n 个互异词条随机生成的二叉搜索树, 平均高度为 $\Theta(\log n)$

✧ n 个互异节点, 在遵守顺序性的前提下, 可随机确定拓扑联接关系



✧ 由 n 个互异节点随机组成的 BST, 若共计 $S(n)$ 棵, 则有

$$S(n) = \sum_{k=1}^n S(k-1) \cdot S(n-k) = \text{catalan}(n) = (2n)! / ((n+1)! / n!)$$

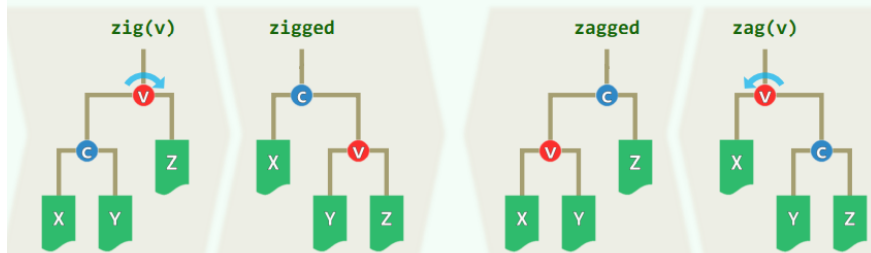
✧ 假定所有 BST 等概率出现, 则其平均高度为 $\Theta(\sqrt{n})$

三是等价 BST 的相互转化问题

等价变换 + 旋转调整

721

❖ 刚刚失衡的BST, 必可**迅速**转换为一棵等价的BBST——为此, 只需 $O(\log n)$ 甚至 $O(1)$ 次旋转



❖ zig和zag: 仅涉及**常数**个节点, 只需调整其间的联接关系; 均属于**局部**的基本操作

❖ 调整之后: v/c深度加/减1, 子(全)树高度的变化幅度, 上下差异**不超过1**

❖ 实际上, 经过**不超过** $O(n)$ 次旋转, 等价的BST均可相互转化(习题解析[7-15])

四是查考固定高度的 AVL 的最少节点数

AVL = 适度平衡

724

❖ 高度为h的AVL树, **至少**包含 $S(h) = \text{fib}(h+3) - 1$ 个节点

为什么?

❖ 固定高度h, 考查节点**最少**的AVL树...

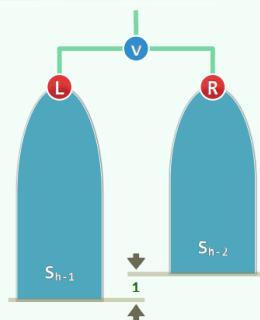
❖ 将其规模记作 $S(h)$

$$S(h) = 1 + S(h-1) + S(h-2)$$

$$S(h) + 1 = [S(h-1) + 1] + [S(h-2) + 1]$$

$$\text{fib}(h+3) = \text{fib}(h+2) + \text{fib}(h+1)$$

❖ 反过来, 由n个节点构成的AVL树, 高度不致超过 $O(\log n)$

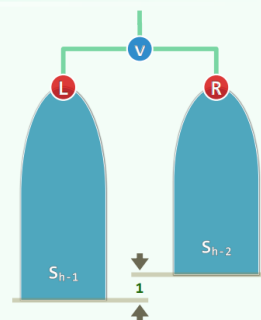
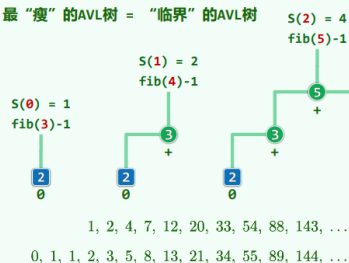


Fibonacci Tree

725

❖ 高度为h, 规模恰为 $S(h) = \text{fib}(h+3) - 1$ 的AVL树

❖ 最“瘦”的AVL树 = “临界”的AVL树



五是考查 AVL 树中任意叶结点深度的结论,也就是习题 7-12

[7-12] 试证明, 在高度为h的AVL树中, 任一叶结点的深度均不小于 $\lceil h/2 \rceil$ 。

【解答】

对树高h做数学归纳。作为归纳基, $h = 1$ 时的情况显然。假设以上命题对高度小于h的AVL树均成立, 以下考查高度为h的AVL树。

根据AVL树的性质, 如图x7.1所示, 此时左、右子树的高度至多为 $h-1$, 至少为 $h-2$ 。

由归纳假设, 在高度为 $h-1$ 的子树内部, 叶节点深度不小于:

$$\lceil (h-1)/2 \rceil$$

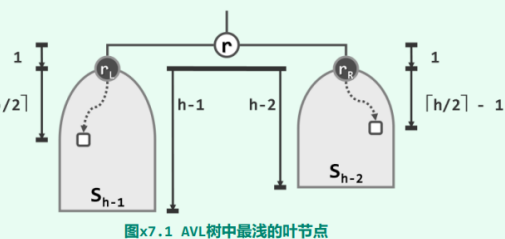
$$\geq \lceil h/2 \rceil - 1$$

而在高度为 $h-2$ 的子树内部, 叶节点深度也不小于:

$$\lceil h/2 \rceil - 1$$

因此在全树中, 任何叶节点深度都不致小于:

$$1 + (\lceil h/2 \rceil - 1) = \lceil h/2 \rceil$$



图x7.1 AVL树中最浅的叶节点

高级搜索树

B 树

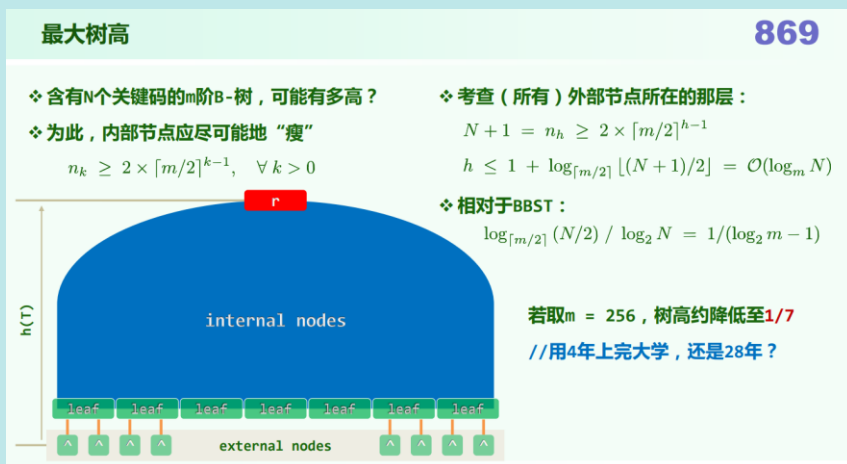
1. 搜索 7 阶 B 树的第 2016 个关键字, 假设 B 树根节点在内存中, 则最多共需启动 () 次 I/O. (2017)

(A)4 (B)5 (C)6 (D)7 (E)8

解析: 根据公式:

$$\log_m(n+1) \leq h \leq 1 + \log_{\lceil \frac{m}{2} \rceil} \left\lceil \frac{n+1}{2} \right\rceil \Rightarrow h \leq 1 + \log_4 1008 < 1 + \log_4 1024 = 1 + 5 \Rightarrow h = 5$$

注意: 外部节点不参与计算 (关键码的数量是 n , 外部节点不包括这个关键码信息的), B 树的高是算了外部节点的, 找到外部节点的时候是不需要 IO 的, 所以最多 IO 的次数是 4 次。



2. 一个具有 2017 个关键码的 7 阶 B 树, 根节点常驻内存, 则一次查找最多进行 () 次 I/O 操作 (2018)

(A)7 (B)6 (C)5 (D)4

解析:

$$\log_m(n+1) \leq h \leq 1 + \log_{\lceil \frac{m}{2} \rceil} \left\lceil \frac{n+1}{2} \right\rceil \Rightarrow h \leq 1 + \log_4 1008 < 1 + \log_4 1024 = 1 + 5 \Rightarrow h = 5$$

同第 1 题

3. 在有 400 个关键码的 20 阶 B 树查找, 最大需要和 39 个关键码进行比较 (2022)

解析:

我们先来计算 B 树的高度, 根据公式

$$\log_m(n+1) \leq h \leq 1 + \log_{\lceil \frac{m}{2} \rceil} \left\lceil \frac{n+1}{2} \right\rceil$$

$\log_{20} 401 \leq h \leq 1 + \log_{10} 200 < 1 + 3 = 4$, 所以 B 树的高度就只能是 3, 第一层在 B 树的根节点内进行查找. 第二层和第三层都是在最右边的节点内进行查找, 节点内部查找是用顺序查找的。

那么问题转化为根结点最多可以有多少个关键码, 我们不妨做一个极端的假设, 假设根节点只有

一个关键码,那么第二层有两个节点,除了根节点之外,还有 398 个关键码,要铺满整个第二层和第三层(指的是节点铺满,不是说节点内的关键码数量都达到最大值),是没有问题的,关键码数量完全够用。

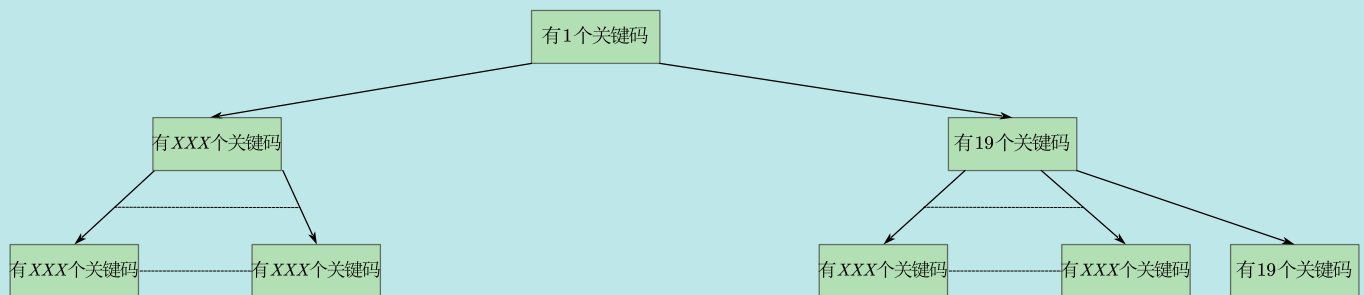
如果根节点包含 19 个关键码,第二层有 20 个节点,每个节点至少要有 9 个关键码,那么第三层至少有 200 个节点,每个节点至少 9 个关键码,目前为止已经有 $19 + 20 \times 9 + 200 \times 9 = 1999$ 个关键码,远超 400 了.所以这种情况是不可能的。

也就是说要找到根节点的最大的关键码的数量,使得第二层和第三层除了最右边的节点之外都包括尽量少的关键码.这个尽量少不一定是最少,因为那样不一定就凑巧满足关键码总数的(视频课中会详细解释.)

不妨设根节点包括 k 个关键码,那么一定要满足关系式如下关系式:(视频里会详细讲怎么来的)

$$k + 9k + 19 + 10k \times 9 + 19 \times 9 + 19 = 100k + 209 \leq 400$$

也就是说 k 的最大值是 1.那我们看看 $k = 1$ 的时候到底行不行呢.是可行的,如图



这个所谓的 k 的最大值是 1,并不是说对于 B 树来说 k 只能是 1,对于这个题的 B 树来说 $k=2$ 也行,但是那样无法保证右侧的节点内的关键码数量达到最多。比如 $k=2$,

第一层有 2 个关键码

第二层有 3 个节点,左边 2 个节点包括 9 个关键码,最右边 1 个节点包括 19 个关键码,第二层有 37 个关键码。

第三层有 40 个节点,左边 39 个节点有 9 个关键码,最右边一个节点包括 19 个关键码,第三层有 370 个关键码,一共有 $2+37+370=409$ 个关键码了,超过了 400 了,所以第二层最右侧的节点关键码数量不能是 19,得是 18,这样也是和 $2+18+19=39$ 个关键码进行比较。

伸展树

4.理想随机的伸展树插入操作的分摊时间复杂度 $O(\log n)$ (✓) (2018)

解析: 理想随机情况下插入,删除,查找的分摊时间复杂度都是 $O(\log n)$

综合评价

843

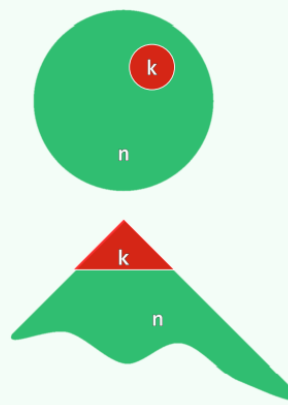
❖ 无需记录节点高度或平衡因子; 编程实现简单易行——优于AVL树

分摊复杂度 $O(\log n)$ ——与AVL树相当❖ 局部性强、缓存命中率极高时 (即 $k \ll n \ll m$)- 效率甚至可以更高——自适应的 $O(\log k)$ - 任何连续的 m 次查找, 都可在 $O(m \log k + n \log n)$ 时间内完成

❖ 若反复地顺序访问任一子集, 分摊成本仅为常数

❖ 不能杜绝单次最坏情况的出现, 不适用于对效率敏感的场所

❖ 复杂度的分析稍嫌复杂——好在有初等的证明...

5. 如果访问顺序不具有局部性, 则伸展树不能保证分摊 $O(\lg n)$ 的性能。 (×) (2019)解析: 如果考虑局部性, 伸展树的分摊时间复杂度是 $O(1)$, 因为总体是 $O(m \lg k + n \lg n)$, 同第 4 题。6. 伸展树总体势能不可能超过 $O(n \log n)$ (✓) (2022)

解析: 单链的时候势能最大

S 的势能

845

❖ (任何时刻的) 任何一棵伸展树 S , 都可以假想地被认为具有势能

$$\Phi(S) = \log \left(\prod_{v \in S} \text{size}(v) \right) = \sum_{v \in S} \log(\text{size}(v)) = \sum_{v \in S} \text{rank}(v)$$

❖ 直觉: 越平衡/倾侧的树, 势能越小/大

- 单链: $\Phi(S) = \log n! = O(n \log n)$ - 满树: $\Phi(S) = \log \prod_{d=0}^h (2^{h-d+1} - 1)^{2^d} \leq \log \prod_{d=0}^h (2^{h-d+1})^{2^d}$

$$= \log \prod_{d=0}^h 2^{(h-d+1) \cdot 2^d} = \sum_{d=0}^h (h-d+1) \cdot 2^d = (h+1) \cdot \sum_{d=0}^h 2^d - \sum_{d=0}^h d \cdot 2^d$$

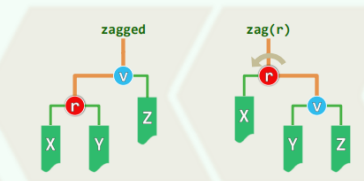
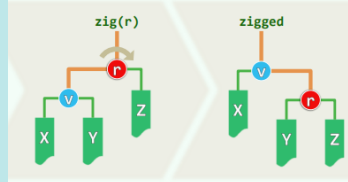
$$= (h+1) \cdot (2^{h+1} - 1) - [(h-1) \cdot 2^{h+1} + 2] = 2^{h+2} - h - 3 = O(n)$$

7. 一棵伸展树经过访问后, 节点被转换到根部, 树的高度不一定降低 () (2022)

解析: 不一定降低, 可能高度相同, 也可能增高

zig / zag

835

❖ 要是 v 只有父亲, 没有祖父呢?❖ 此时必有 $v.\text{parent}() == T.\text{root}()$ ❖ 只需做单次旋转: $\text{zig}(r)$ 或 $\text{zag}(r)$

❖ 好在, 这种情况至多 (在最后) 出现一次

8. 一个伸展树中所有节点都是偶数度, 最深的叶节点需要经过 11 条边才能到达树根, 则该二叉树最少有 ____ 个节点 (根据 2022 考场上的小伙伴回忆说可以非常肯定原题肯定不是这个, 具体不知道要考什么, 有知道的小伙伴可以告诉我下, 我谢谢你了.)

解析: 如果确实是这道题, 可能极其复杂, 并不是无脑 $11 \times 2 + 1 = 23$ 那么简单

9. 如果访问序列不满足局部性, Splay 树未必还能保证分摊 $O(\log n)$ 的时间性能. () (2023)

解析: 如果考虑局部性, 伸展树的分摊时间复杂度是 $O(1)$, 因为总体是 $O(m \lg k + n \lg n)$, 同第 4 题.

红黑树

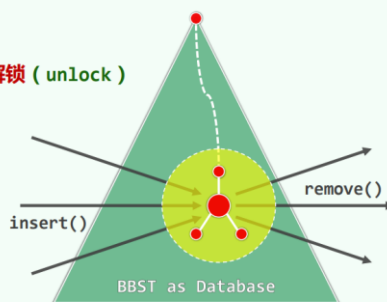
10. 找一个应用场景, 举例说明红黑树有 AVL 树不具有的优势 (2019)

解析: 相比 AVL 树, 红黑树每次插入删除操作所进行的旋转操作都是 $O(1)$ 的, 而 AVL 的删除操作是 $O(\lg n)$ 的。因此由于红黑树的拓扑结构变化非常小, 在并发性、历史版本的维护上具有优势, 可以最大可能的节省空间。应用场景可以举加锁解锁的例子

并发性

892

- ❖ Concurrent access to a database
- ❖ 凡需修改之处都需先加锁 (lock); 修改完成后方予解锁 (unlock)
- 访问延迟主要取决于 “lock/unlock” 周期
- ❖ 对于 BST 而言, 每次修改过程中, 唯
结构有变 (reconstruction) 处才需加锁
- 访问延迟主要取决于这类局部之数量...
- ❖ Splay: 结构变化剧烈, 最差可达 $O(n)$
- ❖ AVL: remove() 时 $O(\log n)$ —— 尽管 insert() 时可保证 $O(1)$
- ❖ Red-Black: 无论 insert/remove, 均不超过 $O(1)$



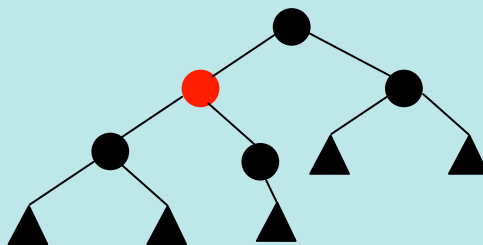
11. 红黑树中所有节点的黑深度和黑高度之和相等. (X) (2020)

解析: 红色节点的黑深度和黑高度的和比黑色节点的黑深度和黑高度的和少 1.

由此可知, 在从根节点通往任一节点的沿途, 黑节点都不少于红节点。除去根节点本身, 沿途所经黑节点的总数称作该节点的黑深度 (black depth) —— 根节点的黑深度为 0, 其余依此类推。故条件(4)亦可等效地理解和描述为 “所有外部节点的黑深度统一”。

由条件(4)可进一步推知, 在从任一节点通往其任一后代外部节点的沿途, 黑节点的总数亦必相等。除去 (黑色) 外部节点, 沿途所经黑节点的总数称作该节点的黑高度 (black height)。如此, 所有外部节点的黑高度均为 0, 其余依此类推。

特别地, 根节点的黑高度亦称作全树的黑高度, 在数值上与外部节点的黑深度相等。



12.若 rb-tree 删除导致双黑修正进行了 $\Omega(\log n)$ 次重染色,则至少旋转一次 (×) (2022)

解析: 有可能不发生旋转,最后把根节点

BB-2B: s 为黑, 且两个孩子均为黑; p 为黑 926

- ❖ s 转红; r 与 p 保持黑
- ❖ 红黑树性质在局部得以恢复
- ❖ 在对应的 B-树中, 等效于下溢节点与兄弟合并
- ❖ 合并之前, p 和 s 均对应于单关键字节点
- ❖ 失去关键字 p 后, 上层节点必然继而下溢
- ❖ 好在可继续分情况处理

高度递增, 至多 $O(\log n)$ 层 (步)

	旋转	染色	此后
(1) 黑s有红子t	1~2	3	调整随即完成
(2R) 黑s无红子, p红	0	2	调整随即完成
(2B) 黑s无红子, p黑	0	1	必再次双黑, 但将上升一层
(3) 红s	1	2	转为(1)或(2R)

如果最后一次做 BB-2B 的操作, P 还是根, 那么刚好调整完成, 没有发生过旋转.

13.红黑树一条路径上有 5 个红节点, 则该树最少有 ____ 个节点 (2022)

解析: T_n 表示有 n 个节点的纯黑的红黑树.

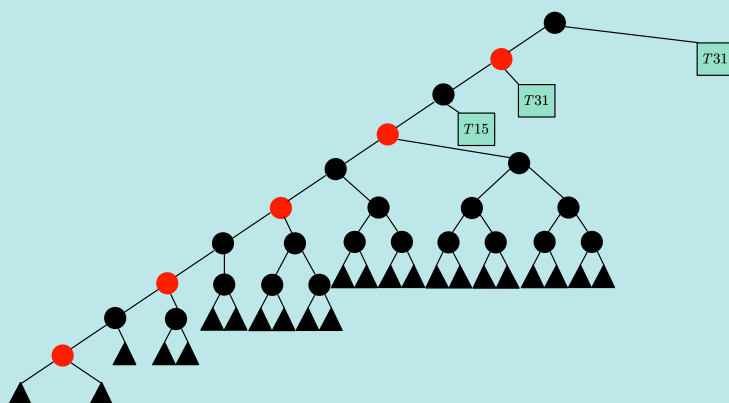
先算出来整棵树的黑高度最小情况是多少, 也就是 5 (红黑相间的情况), 注意: 黑高度的定义. 是要除去外部节点的.

然后先算左侧的路径节点的总数是 11 (6 黑 5 红)

然后计算其他结点 $31+31+15+15+7+7+3+3+1+1=114$, 一共是 125 个节点

用 $S(n)$ 表示一条路径上有 n 个红色节点的红黑树的最少的节点数, 则

$$S(n) = 2n + 1 + \sum_{k=1}^n 2(2^k - 1) = 2^{n+2} - 3 \quad (n = \text{红色节点个数} = \text{树的黑高度})$$



14. 规模同为 2023 的 AVL 树和红黑树各做一次搜索, 在最坏情况下后者的搜索长度比前者更长 (✓)
(2023)

解析: 个人认为这个规模 2023 对于红黑树而言指的是关键字 (也就是不算外部节点.)

根据习题解析 8-13

[8-13] 考查含有 2012 个内部节点的红黑树。

a) 该树可能的最小黑高度 $d_{\min}$ 是多少?

N 个内部节点的红黑树的最大高度 (计入外部节点之后的高度) 为

$$h_{\max} = \max \left\{ 2(\lfloor \log_2 (N+2) \rfloor - 1), 2 \left\lfloor \log_2 \frac{N+2}{3} \right\rfloor + 1 \right\}, \text{ 代入 } N = 2023$$

$$h_{\max} = \max \{18, 19\} = 19$$

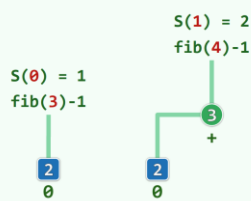
用做搜索的高度为 $19-1=18$ (因为再计算搜索长度时不考虑外部节点)

Fibonacci Tree

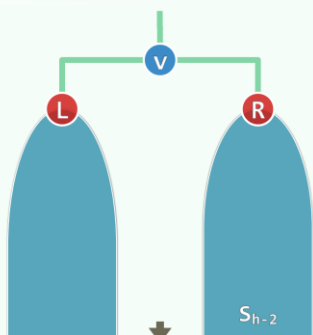
725

✧ 高度为 h , 规模恰为 $S(h) = \text{fib}(h+3) - 1$ 的 AVL 树

✧ 最“瘦”的 AVL 树 = “临界”的 AVL 树



$$S(2) = 4 \\ \text{fib}(5) - 1$$



$$S(h) = \text{fib}(h+3) - 1$$

$$\begin{array}{cccccccc} \text{fib}(0) = 0 & \text{fib}(1) = 1 & \text{fib}(2) = 1 & \text{fib}(3) = 2 & \text{fib}(4) = 3 & \text{fib}(5) = 5 & \text{fib}(6) = 8 \\ \text{fib}(7) = 13 & \text{fib}(8) = 21 & \text{fib}(9) = 34 & \text{fib}(10) = 55 & \text{fib}(11) = 89 & \text{fib}(12) = 144 & \text{fib}(13) = 233 \\ \text{fib}(14) = 377 & \text{fib}(15) = 610 & \text{fib}(16) = 987 & \text{fib}(17) = 1597 & \text{fib}(18) = 2584 & & \end{array}$$

$$S(15) = \text{fib}(17) - 1 = 2583 > 2023, S(14) = \text{fib}(16) - 1 = 1596, \text{ 所以高度为 } 14$$

本章真题总结:

高级搜索树这里主要题型有以下几种:

一是考 B 树的 IO 最大次数或者比较次数 (比较次数比较难), IO 次数就是送分题。

二是伸展树分摊时间复杂度问题

三是伸展树伸展的细节问题 (2022 真题, 不知道是怎么考的)

四是考红黑树的特点.

五是考红黑树插入删除的结论 (2022 真题, 主要就是记住下表)

复杂度 914

❖ 重构、染色均属常数时间的局部操作，故只需统计其总次数

❖ 红黑树的每一次插入操作，都可在 $O(\log n)$ 时间内完成

❖ 其中至多做 $O(\log n)$ 次节点染色、1次“3+4”重构

	旋转	染色	此后
u为黑	1~2	2	调整随即完成
u为红	0	3	可能再次双红，但必上升两层

复杂度 930

❖ 红黑树的每一删除操作

- 至多做 $O(\log n)$ 次重染色
- 1次“3+4”重构、1次单旋

故必可在 $O(\log n)$ 时间内完成

	旋转	染色	此后
(1) 黑s有红子t	1~2	3	调整随即完成
(2R) 黑s无红子, p红	0	2	调整随即完成
(2B) 黑s无红子, p黑	0	1	必再次双黑, 但将上升一层
(3) 红s	1	2	转为(1)或(2R)

六是 AVL 和红黑树的搜索长度的比较的问题（2023）。

可能会考,但是真题尚未考过的题型（涉及算法题的放到最后算法题的部分）：

一是 B 树的插入和删除的具体过程

二是 B 树这里在习题解析中的小结论

三是伸展树的插入和删除的优化的细节

四是红黑树的插入和删除的细节问题。（2022 考过一个判断题,但感觉考的不是很细）

散列

1. 原题数据没回忆起来, 故用殷人昆的习题解析中的一道题代替。(2016)

散列表长为 13, 采用双散列函数解决冲突: $H(\text{key}) = \text{key} \% 13$, $H'(\text{key}) = (7 \times \text{key} \% 10) + 1$ 。

输入顺序为: 12, 23, 45, 57, 20, 3, 78, 31, 15, 36。

1) 构造散列表

2) 求等概率下搜索成功的平均查找长度

要注意%运算和 $\times$, $\div$ 运算的优先级是一样的。

解析:

(1)

12 插入的位置是 12

23 插入的位置是 10

45 插入的位置是 6

57 插入的位置是 5

20 插入的位置是 7

3 插入的位置是 3

78 插入的位置是 0

31 插入的位置是 5, 和 57 冲突第 1 次, 所以试探 $[H(\text{key}) + 1 \times H'(\text{key})] \% 13 = (5 + 1 \times 8) \% 13 = 0$, 和 78 第 2 次冲突, 所以试探 $[H(\text{key}) + 1 \times H'(\text{key})] \% 13 = (5 + 2 \times 8) \% 13 = 8$, 所以 31 插入的位置是 8

15 插入的位置是 2

36 插入的位置是 10, 和 23 第 1 次冲突, 所以试探 $[H(\text{key}) + 1 \times H'(\text{key})] \% 13 = (10 + 1 \times 3) \% 13 = 0$ 和 78 第 2 次冲突, 所以试探 $[H(\text{key}) + 2 \times H'(\text{key})] \% 13 = (10 + 2 \times 3) \% 13 = 3$, 和 3 第 3 次冲突, 所以试探 $[H(\text{key}) + 3 \times H'(\text{key})] \% 13 = (10 + 3 \times 3) \% 13 = 6$, 和 45 第 4 次冲突, 所以试探 $[H(\text{key}) + 4 \times H'(\text{key})] \% 13 = (10 + 4 \times 3) \% 13 = 9$ 散列表如下:

0	1	2	3	4	5	6	7	8	9	10	11	12
78		15	3		57	45	20	31	36	23		12

(2) $ASL = (1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 3 + 1 + 5) / 10 = 1.6$

2. 若 M 取不超过散列表长度的素数, 则存储关键码不能保证其分布均匀。(✓) (2017)

解析:

题目也没说散列函数是怎么构造的, 也没说怎么处理冲突, 不过没关系, 假设用的除余法吧。

hash(Key) 等于 $0 \sim M-1$ 的概率是相等的。关键码的分布是均匀的。

如果取的的就是表长,比如 $M=7$,我们用除余法,等于 $0,1,2,3,4,5,6$ 的概率都是 $1/7$

如果散列表长是 7,但是我们取这个 $M=5$,我们用除余法,等于 $0,1,2,3,4$ 的概率都是 $1/5$,只有冲突了之后,才有可能分布到 6,7 的位置。

考虑极端情况,表长是 100, $M=5$

就插入 5 个关键码,那么分布会极其不均匀,比如采用线性试探,那么大于 10 的位置是完全没机会插入数据的。

3. 对长度为 $m=4k+3$ 素数的散列表双平方探测一定能访问其全部元素 (✓) (2018)

解析: 对,老生常谈的问题了.

$4k + 3$

979

❖ 两类素数: $3 \quad 5 \quad 7 \quad 11 \quad 13 \quad 17 \quad 19 \quad 23 \quad 29 \quad 31$

❖ 表长取作素数 $M = 4 \cdot k + 3$, 即必然可以保证查找链的前 M 项均互异

$\pm i^2$	-36	-25	-16	-9	-4	-1	0	1	4	9	16	25	36
M	5				1	4	0	1	4				
7				5	3	6	0	1	4	2			
11		8	6	2	7	10	0	1	4	9	5	3	
13	3	1	10	4	9	12	0	1	4	9	3	12	10

4. 散列长为 2017, 采用单平方探测, 已经存入 1000 个元素, 问此时最多有 () 个懒惰删除的桶单元

(A)8

(B)9

(C)1016

(D)1017

解析: 题目有些模糊

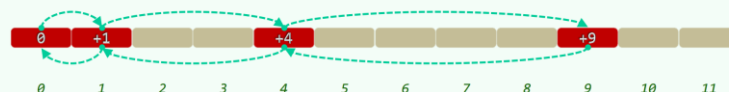
如果按照 $\left\lfloor \frac{M}{2} \right\rfloor$ 来看, 平方探测可以插入 $\left\lfloor \frac{M}{2} \right\rfloor = 1009$, 所以最多插入 1009 个元素, 现在有 1000, 最多懒惰删除了 9 个, 就是选 B, 我也一度觉得这个题就选 B, 但是这得改查找的算法 (如何判断要找的元素不在散列表中), 这是相当没必要的, 也不是默认的情况。 (视频中会详细解释这一点)

装填因子, 须足够小!

974

❖ $\{0, 1, 2, 3, 4, 5, \dots\}^2 \% 12 = \{0, 1, 4, 9\}$

❖ 若为合数: $n^2 \% M$ 可能的取值可能少于 $\lceil M/2 \rceil$ 种——此时, 只要对应的桶均非空...



❖ $\{0, 1, 2, 3, 4, 5, \dots\}^2 \% 11 = \{0, 1, 4, 9, 5, 3\}$

❖ 若为素数: $n^2 \% M$ 可能的取值恰好会有 $\lceil M/2 \rceil$ 种, 且由查找链的前 $\lceil M/2 \rceil$ 项取遍

❖ 定理: 若 M 是素数, 且 $\lambda \leq 0.5$, 就一定能够找出; 否则, 不见得

所以得按照 $\lambda \leq 0.5$ 来看, 也就是说应该选 A

5.与开放散列相比,封闭散列有什么优点? 试举例说明两点 (2019)

解析: (1) 就地解决,不需要额外的空间(多槽位,独立链,公共溢出区都需要额外的空间)
(2) 可以更充分地利用缓存.

6.开放式散列与封闭式散列更可以有效利用局部缓存. (×) (2020)

解析: 封闭散列在散列表内部解决冲突,更能充分利用局部缓存

7.在 n 个节点跳转表中,单个词条的期望塔高是 $\Theta(\log n)$ (×) (2020)

解析: 单个词条的期望塔高是 2,整体的期望的塔高才是 $\Theta(\log n)$,跳转表查找过程中横向跳转和纵向跳转的次数都是 expected- $O(\log n)$,总体的时间为 expected- $O(\log n)$

8.一个初始为空容量为 $M=13$ 的哈希表 H ,采用除留余数法寻址,采用双向平方试探法解决冲突,现插入如下数据,按格式填写下表(空桶不做标记) (2022)

解析:

H[]	0	1	2	3	4	5	6	7	8	9	10	11	12
插入 44						44							
插入 35						44				35			
插入 18						44	18			35			
插入 58						44	18	58		35			
插入 71						44	18	58		35	71		
插入 32			32			44	18	58		35	71		
插入 84			32	84		44	18	58		35	71		

如果此时插入 2021 会有什么效果? 为什么?

效果就是压根插不进去,理由如下

0,1,4,9 对 13 的余数是 0,1,4,9

16 对 13 的余数是 3

25 对 13 的余数是 12

36 对 13 的余数是 0

49 对 13 的余数是 10

64 对 13 的余数是 12

81 对 13 的余数是 3

100 对 13 的余数是 9

121 对 13 的余数是 4

144 对 13 的余数是 3

$0^2, 1^2, \dots, 12^2$ 穷举同余类就只有 0,1,3,4,9,10,12

$(13k+i)^2 = (13k)^2 + 2 \times 13ki + i^2$ 和 i^2 同余因此对于 $k^2 (k=0,1,2,\dots)$,

k^2 对 13 的全部同余类是 0,1,3,4,9,10,12 而且还对称 $4+9=13, 3+10=13, 1+12=13$

故 $\pm k^2$ 对 13 的全部同余类是 0,1,3,4,9,10,12

$6 \pm k^2$ 对 13 的全部同余类是 6,7,9,10,2,3,5 这 7 个

刚好 32,84,44,18,58,35,71 刚好把这 7 个位置都给占了。

9. 有 n 个词条的跳转表 (skiplist) 所占用的空间了, 在极端情况下会大于 $\Omega(n^2)$ (✓) (2023)

解析:

极端情况确实如此, 极端情况 h 本身都可能是 $\Omega(n^2)$, 只是概率极低而已。

9.2.5 空间复杂度

■ “生长概率逐层减半” 条件

不难理解, 其中各塔高度的随机分布规律 (如最大值、平均值等), 对跳转表的总体性能至关重要。反之, 若不就此作出显式的限定, 则跳转表的时间和空间效率都难以保证。

比如, 若将最大塔高 (亦即跳转表的层高) 记作 h , 则在极端情况下, 每个词条所对应塔的高度均有可能接近甚至达到 h 。果真如此, 在查找及更新过程中需要访问的节点数量将难以控制, 时间效率注定会十分低下。同时, 若词条总数为 n , 则在此类情况下, 跳转表所需的存储空间量也将高达 $\Omega(nh)$ 。

然而幸运的是, 若能采用简明而精妙的策略, 控制跳转表的生长过程, 则在时间和空间方面都可实现足够高的效率。就效果而言, 此类控制策略必须满足所谓 “生长概率逐层减半” 条件:

对于任意 $0 \leq k < h$, S_k 中任一节点在 S_{k+1} 中依然出现的概率, 始终为 $1/2$

10. 采用独立链 (Scparate chaining) 法的某散列表长度为 119, 在存在 2023 个词条时, 一次成功查找的过程中所需试探次数的期望值是 9。(2023)

解析: 每条独立链期望的长度为 $\frac{2023}{119} = 17$, 试探次数的期望为 $\frac{17+1}{2} = 9$

本章真题总结:

散列这里主要题型有以下几种:

一是给一堆数, 告诉你散列函数和解决冲突的方法, 然后让你往散列表里面插, 完了还让求平均查找长度, 基本题型, 老老实实插进去就行了。

二是证明插不进去了, 通常是平方探测或者双平方探测的情况, 和习题解析的套路差不多 (坦率的讲我是没想到真会这么考的, 但是 2022 真题就是考了)

三是开散列和闭散列的特点。

四是跳转表的塔高, 要注意区分整个跳转表的塔高还是单个词条的塔高。

五是考了平方探测和双平方探测的特点,ppt 里面也都有,不是难点

六是 2018 真题稍微提了一下懒惰删除.但是主要考查的并不是懒惰删除.

七是考了跳转表 (skiplist) 所占用的空间

八是考了独立链的散列表的试探次数的期望.

可能会考,但是真题尚未考过的题型 (涉及算法题的放到最后算法题的部分):

一是三种开散列 (多槽位,独立链,公共溢出区) 的特点

二是重散列,注意它和双散列的区别

三是关于懒惰删除的问题,把习题解析 9-16 弄清楚.

四是跳转表的查找问题,ppt 里的结论弄清楚.

优先级队列

1. 输入随机的情况下完全二叉堆的插入平均时间是常数 (✓) (2018)

解析: 老生常谈的问题了, 插入的平均情况下是 $O(1)$, 删除仍然是 $O(\log n)$

2. 一个右侧路径长度为 k 的左式堆, 其顶点数量 () 为 (2018)

(A). 至少: 2^k (B) 至少: $2^{k+1} - 1$ (C) 至多: 2^k (D) 至多: $2^{k+1} - 1$

解析: 右侧链固定的情况下, 至少有 $2^{k+1} - 1$ 个节点, 有 $2^k - 1$ 个内部节点, 没有至多

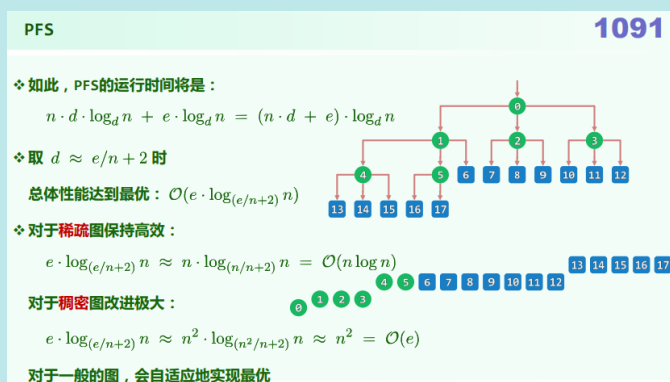


3. 在理想随机情况下, 完全二叉堆插入操作平均只需 $O(1)$, 尽管最坏情况下为 $O(\log(n))$ 。(✓) (2019)

解析: 插入平均是 $O(1)$, 最差情况是 $O(\log n)$, 删除的平均是 $O(\log n)$, 最差是 $O(\log n)$

4. Dijkstra 用于稠密图时, 为何使用多叉堆替换掉完全二叉堆? 多叉堆的分叉数怎么确定? (2019)

解析: 使用完全二叉堆的情况下, PFS 总体运行时间为 $O[(n+e)\log n]$, 对于稠密图反而不如常规实现版本。采用多叉堆之后, PFS 的运行时间为 $O[(n+d)\log_d n]$, 取 $d = \frac{e}{n} + 2$, 对于稠密图有较大改进 $O(n^2)$



5. 相比锦标赛树, 败者树的优势是什么? (2019)

解析: 每次比较只需要和父亲节点相比, 而锦标赛树需要同兄弟比较并写入父节点。败者树只需要在与父亲节点比较失败才需要写入。在时间复杂度常系数上优于锦标赛树。

6. 败者树删除的时间复杂度在常系数上优于胜者树。(✓) (2020)

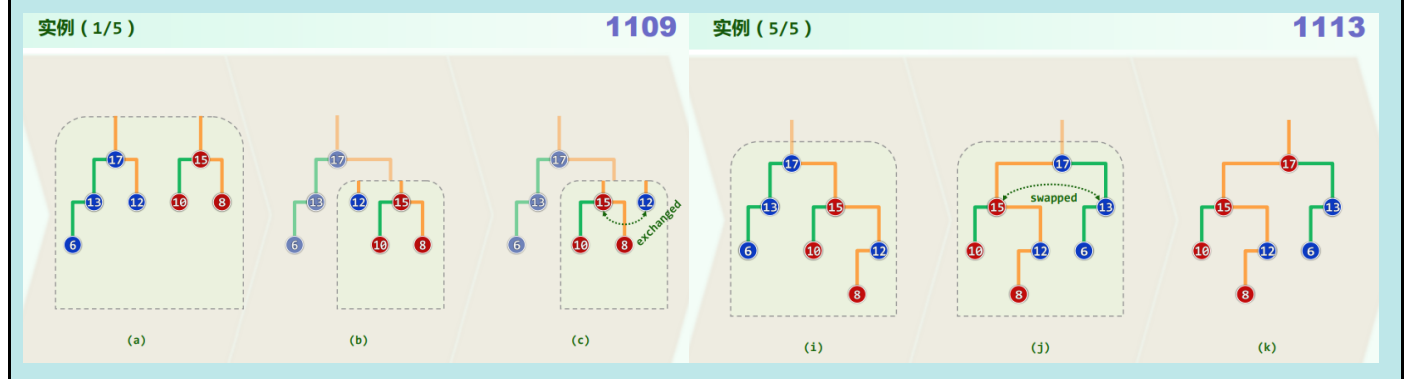
解析: 胜者树需要交替访问沿途节点及其兄弟, 败者树可以避免这样的迂回, 常系数上优于胜者树

7. 完全二叉堆删除元素在最坏情况下时间复杂度为 $O(\log n)$, 但是平均情况下时间复杂度为 $O(1)$ (×) (2020)

解析: 老生常谈了, 插入的平均情况下才是 $O(1)$, 删除仍然是 $O(\log n)$

8. 采用 crane 算法将左式堆 A 和 B 合并为左式堆 H, 则 H 右侧链上的节点未必都来自 A 或者 B 的右侧链。 (√) (2020)

解析: 因为要满足左式堆的性质 $npl(x-lc) \geq npl(x-rc)$, 因此最后合并的左式堆, 有可能左右子树交换。这样会使得右侧链上的节点未必都来自 A 或者 B 的右侧链。

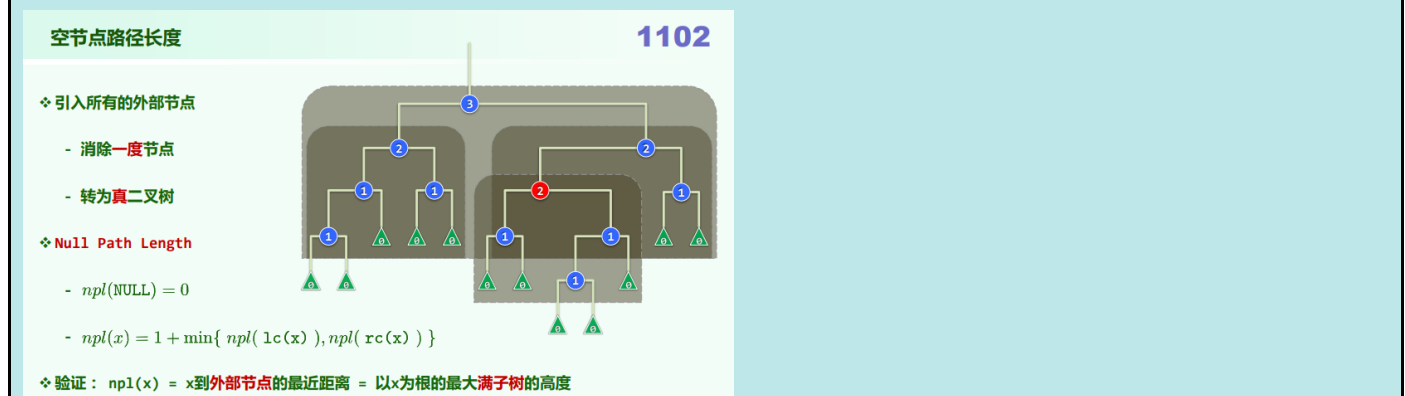


9. 建立一个完全二叉堆时间复杂度为 $O(n \log n)$ 。 (×) (2020)

解析: Floyd 建堆的时间为 $O(n)$

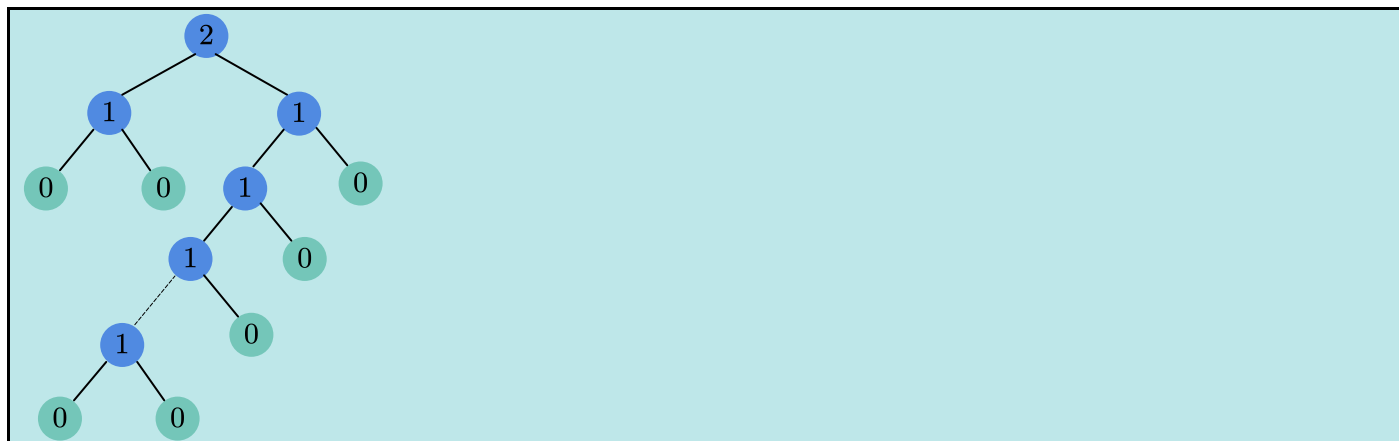
10. 左式堆兄弟子堆满足 $L.height \geq R.height$ (×) (2022)

解析: 比如红色的 2 为根的左式堆, 右子堆就更高。



11. 在包含 2023 个节点的左式堆中, 右子堆最多可能有 2019 个节点 (2023)

解析: 2019 个, 要注意外部节点的问题。



12. 计算 (10 分) (2023)

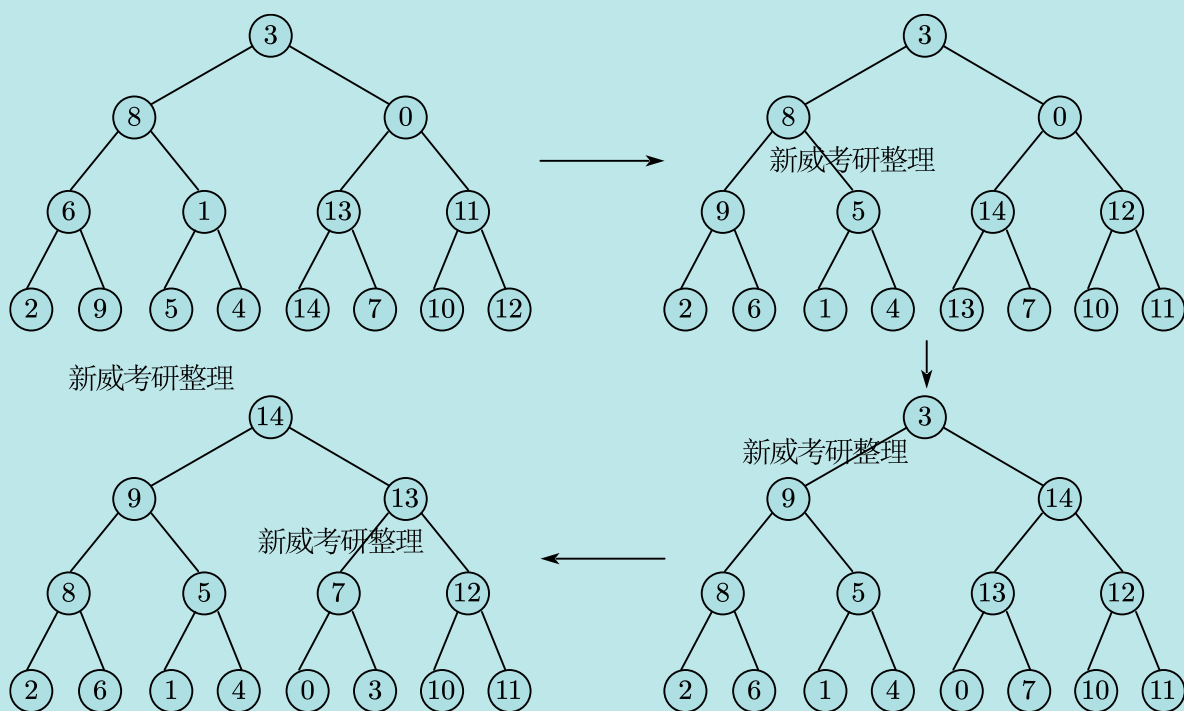
试采用 Floyd 算法将以下向量转换为大顶堆, 并按如下格式, 依次给出向量在各次下滤之后的内容:

K	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
向量 A[k]	3	8	0	6	1	13	11	2	9	5	4	14	7	10	12
第 1 次下滤后	3	8	0	9	5	14	12	2	6	1	4	13	7	10	11
第 2 次下滤后	3	9	14	8	5	13	12	2	6	1	4	0	7	10	11
第 3 次下滤后	14	9	13	8	5	7	12	2	6	1	4	0	3	10	11

解析:

```
template <typename T> void heapify( T* A, const Rank n ) { //Floyd建堆算法, O(n)时间
    for ( Rank i = n / 2 - 1; - 1 != i; i-- ) //自底而上, 依次
        percolateDown( A, n, i ); //经下滤合并子堆
}
```

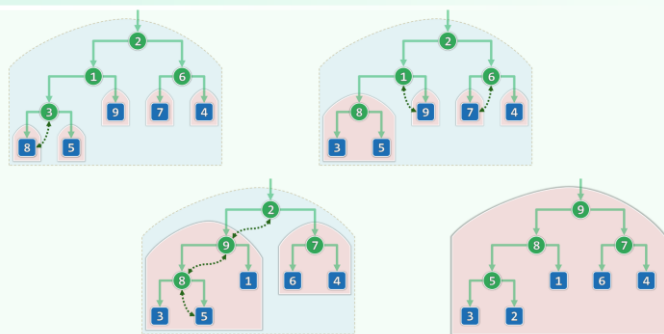
```
1 //对向量前n个词条中的第i个实施下滤, i < n
2 template <typename T> Rank percolateDown( T* A, Rank n, Rank i ) {
3     Rank j; // i及其(至多两个)孩子中, 堪为父者
4     while ( i != ( j = ProperParent( A, n, i ) ) ) //只要i非j, 则
5         swap( A[i], A[j] ), i = j; //二者换位, 并继续考查下降后的i
6     return i; //返回下滤抵达的位置 (亦i亦j)
7 }
```



考试的时候标注好所说的“第几次下滤后”的具体含义,最好把图也给画出来,别只是填空,免得老师有误判

实例

1068



本章真题总结:

优先级队列这里主要题型有以下几种:

一是考完全二叉堆的建堆的时间复杂度,插入和删除的最坏情况下复杂度和平均情况下的复杂度问题,考了好几次.

二是左式堆合并问题,左式堆右侧链长度问题,左式堆左右子堆的高度对比问题.右子堆最大规模.

三是多叉堆去优化 PFS 算法,分支数多少合适的问题.

四是锦标赛树和败者树的问题.

五是建堆的具体过程.

可能会考,但是真题尚未考过的题型 (涉及算法题的放到最后算法题的部分):

一是堆的插入和删除的具体过程

二是可能会考到个别的习题解析的结论.

三是堆排序的具体过程.

串

1. 在字符集各字符出现概率相同时, kmp 算法渐进时间复杂度接近蛮力算法。 (✓) (2017)

解析: 习题解析 11-9, 答案对, 2020 年真题解析也讲过. 注意不要和 11-10 的结论弄混。

11-9 的意思是只要各字符等概率出现, BF 算法的效果就是很好的, 11-10 的意思是等概率的前提下, 字符集越大 BF 算法就越好。

[11-9] 教材 309 页代码 11.1、310 页代码 11.2 所实现的两个蛮力算法, 在通常情况下的效率并不算低。

现假定所有字符出现的概率均等, 试证明:

a) 任意字符比对的成功与失败概率分别为 $1/s$ 和 $(s-1)/s$, 其中 $s = |\Sigma|$ 为字符表的规模;

【解答】

每个字符各有 $1/s$ 的概率出现, 故任何一对字符相同、不同的概率分别为 $1/s$ 和 $(s-1)/s$ 。

b) 在 P 与 T 的每一对齐位置, 需连续执行恰好 k 次字符比对操作的概率为 $(s-1)/s^k$;

【解答】

恰好执行 k 次字符对比, 当且仅当前 k-1 次成功, 但最后一次失败。根据 a) 的分析结论, 这类事件发生的概率应为:

$$(1/s)^{k-1} \cdot (s-1)/s = (s-1)/s^k$$

c) 在 P 与 T 的每一对齐位置, 需连续执行字符比对操作的期望次数不超过 $s/(s-1) \leq 2 = O(1)$ 。

【解答】

由 b) 的分析结论, 每一次字符比对都可视作一次伯努利实验 (Bernoulli trial), 成功与失败的概率分别为 $1/s$ 和 $(s-1)/s$; 而每趟比对的次数 X, 则符合几何分布 (geometric distribution) —— 亦即, 其中前 X-1 次实验成功的概率各为 $1/s$, 最终一次实验失败的概率为 $(s-1)/s$ 。因此, X 的期望值不超过 $s/(s-1)$ 。

直接由期望值的定义出发, 也可得出同样结论。具体地, 连续执行字符比对操作的期望次数, 应该就是所有可能的次数, 关于其对应概率的加权平均, 亦即:

$$\sum_{k=1}^{\infty} k \cdot (s-1)/s^k = (s-1) \cdot \sum_{k=1}^{\infty} k/s^k < (s-1) \cdot \sum_{k=1}^{\infty} k/s^k = s/(s-1)$$

[11-10] BM 算法与 KMP 算法分别擅长于处理何种类型的字符串? 为什么?

【解答】

正如教材第 11.4.5 节所指出的, 在评价不同串匹配算法各自的实用范围时, 在不同应用中单次比对的成功概率, 扮演着重要的角色。而根据习题 [11-9] 的分析结论, 在通常的情况下, 这一概率首先并直接取决于字符集的规模。

当字符集规模较小时, 单次比对的成功概率较高, 蛮力算法的效率较低。此时, KMP 算法稳定的线性复杂度, 更能体现出优势; 而采用 BC 表的 BM 算法, 却并不能大跨度地向前移动。

反之, 若字符集规模较大, 则单次比对的成功概率较小, 蛮力算法也能接近于线性的复杂度。此时, KMP 算法尽管依然保持线性复杂度, 但相对而言的优势并不明显; 而采用 BC 表的 BM 算法, 则会因比对失败的概率增加, 可以大跨度地向前移动。

性能

❖ 空间 = $|bc| + |gs| = O(|\Sigma| + m)$

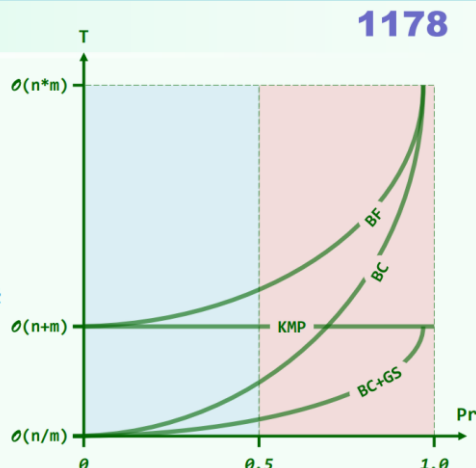
❖ 预处理: $O(|\Sigma| + m)$

❖ 查找效率

- 最好 $O(n/m)$
- 最差 $O(n+m)$ // 参照 KMP 的分析方法

❖ 关键因素

- 单次比对成功的概率
- 通常, $Pr = 1/s$



2. 利用没改进的 next 表, KMP 算法时间复杂度也是 $O(n+m)$ (✓) (2017)

解析: 是的, 不论是不是改进版的 next 表, KMP 算法的时间复杂度都是 $O(n+m)$

3. 对于长度为 m 的随机 0,1 串进行串匹配时, 好后缀数组中 $gs[0]=1$ 的概率为 () (2018)

- (A) $\frac{1}{m}$ (B) $\frac{1}{2^{m-1}}$ (C) $\frac{1}{2^m}$ (D) $\frac{1}{2^{m+1}}$

解析:

要么是:

T	1	0	0	0	0	0	0	0	0
P	0	0	0	0	0	0	0	0	0

要么是:

T	0	1	1	1	1	1	1	1	1
P	1	1	1	1	1	1	1	1	1

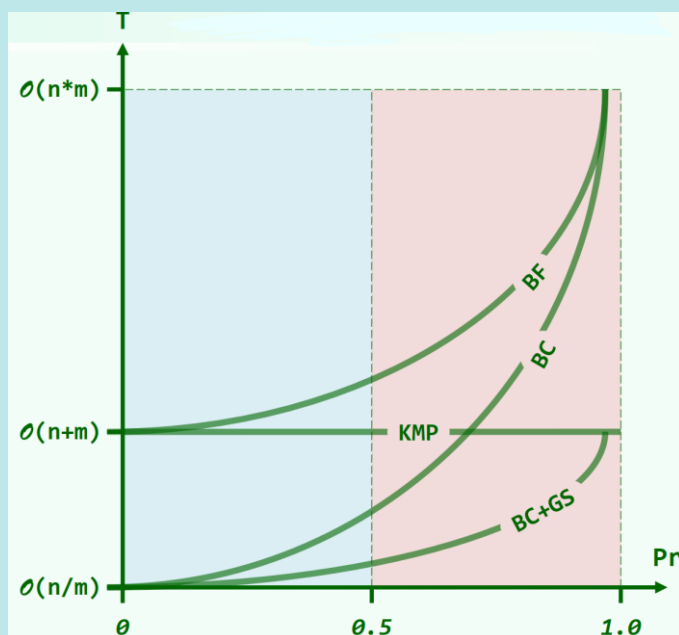
$$P = \frac{1}{2} \cdot \left(\frac{1}{2}\right)^{m-1} + \frac{1}{2} \cdot \left(\frac{1}{2}\right)^{m-1} = \frac{1}{2^{m-1}}$$

4. 未改进过的 next 表, KMP 算法可以在线性时间完成串匹配。 (✓) (2019)

解析: 是的, 不论是不是改进版的 next, KMP 算法的时间复杂度都是 $O(n+m)$

5. KMP 算法相对于蛮力算法的优势在什么条件下足够明显? 为什么? (2019)

解析: 单次匹配概率越大 (字符集越小) 的场合, KMP 的优势越明显; 因为蛮力算法的运行时间随着单次匹配成功率的增加而增加, KMP 的运行时间不随着单次匹配成功率的变化而变化。



要注意单次匹配成功率是可以大于 0.5 的, 比如 0,1 串 0 出现的概率是 0.9 而 1 出现的概率是 0.1 的话, 那么匹配成功的概率就是 $0.9 \times 0.9 + 0.1 \times 0.1 = 0.82$

6. 模式串和文本串均有 26 个大写字母组成, 那么蛮力算法在最好情况下的时间复杂度 () KMP 算法,

解析: s 是字符集大小, 在 P 与 T 的每一对齐位置, 需连续执行字符比对操作的期望次数不超过 $s/(s-1) \leq 2 = O(1)$, 只是最坏的情况下, BF 算法比 KMP 算法渐进意义上要大而已, 最好和平均都一样的。

(C) next[13]=0, next[1]=-1 (D) next[13]=-1, next[1]=-1

本题仅仅是问 $\text{next}[13]=1$, $\text{next}[1]=-1$ 和所以其实可以单独看这两个的,考场上赶时间的话,不一定要把整个 next 表构造出来.

比如上例,P 串是 m 个 a

T 串是 m-1 个 a 一个 c 然后一直这么循环下去,最后 m 个字符是 m 个 a

```

1 int* buildSS ( char* P ) { //构造最大匹配后缀长度表: O(m)
2     int m = strlen ( P ); int* ss = new int[m]; //Suffix Size表
3     ss[m - 1] = m; //对最后一个字符而言, 与之匹配的最长后缀就是整个P串
4     // 以下, 从倒数第二个字符起自右向左扫描P, 依次计算出ss[]其余各项
5     for ( int lo = m - 1, hi = m - 1, j = lo - 1; j >= 0; j -- )
6         if ( ( lo < j ) && ( ss[m - hi + j - 1] < j - lo ) ) //情况一
7             ss[j] = ss[m - hi + j - 1]; //直接利用此前已计算出的ss[]
8         else { //情况二
9             hi = j; lo = min ( lo, hi );
10            while ( ( 0 <= lo ) && ( P[lo] == P[m - hi + lo - 1] ) ) //二重循环?
11                lo--; //逐个对比处于(lo, hi]前端的字符
12            ss[j] = hi - lo;
13        }
14    return ss;
15 }
16
17 int* buildGS ( char* P ) { //构造好后缀位移量表: O(m)
18     int* ss = buildSS ( P ); //Suffix Size table
19     size_t m = strlen ( P ); int* gs = new int[m]; //Good Suffix shift table
20     for ( size_t j = 0; j < m; j ++ ) gs[j] = m; //初始化
21     for ( size_t i = 0, j = m - 1; j < UINT_MAX; j-- ) //逆向逐一扫描各字符P[j]
22         if ( j + 1 == ss[j] ) //若P[0, j] = P[m - j - 1, m), 则
23             while ( i < m - j - 1 ) //对于P[m - j - 1]左侧的每个字符P[i]而言 (二重循环?)
24                 gs[i++] = m - j - 1; //m - j - 1都是gs[i]的一种选择
25     for ( size_t j = 0; j < m - 1; j ++ ) //画家算法: 正向扫描P[]各字符, gs[j]不断递减, 直至最小
26         gs[m - ss[j] - 1] = m - j - 1; //m - j - 1必是其gs[m - ss[j] - 1]值的一种选择
27     delete [] ss; return gs;
28 }

```

rank	0	1	2	...	m-2	m-1
P[]	a	a	a	...	a	a
ss[]	1	2	3	...	m-1	m
gs[]	1	2	3	...	m-1	m

gs[]初始化成 m(对应代码 20 行)

一直都是 $ss[j] == j+1$ 的情况(22 行的判断恒为真)

一开始 $j == m-1$, 进不去 23 行的 while 循环

然后 $j--$, 这时候 $j == m-2$, 进入第 23 行的 while 循环, 更新了 $gs[0] = 1$

然后 $j--$, 这时候 $j == m-3$, 进入第 23 行的 while 循环, 更新了 $gs[1] = 2$

...

然后 $j--$, 这时候 $j == 0$, 进入第 23 行的 while 循环, 更新了 $gs[m-2] = m-1$

最后画家算法反复地做 $gs[m-j-2] = m-j-1$ 的操作. 相当于什么都没改变.

这样的情况下 $gs[m-1] == m$

所以每次 a 和 c 匹配不成功的时候, P 串都右移动 m 个字符直到最后匹配成功, 时间复杂度是 $O(n/m)$

10.kr 算法在匹配失败后无法在 $O(1)$ 内找到下一个(×)(2022)

回忆版原题不太通顺改题目如下:

kr 算法在一次匹配指纹失败后, 无法在 $O(1)$ 内找到下一个指纹(×)(2022)

解析:

快速指纹更新

最后, 讨论快速指纹计算的实现。对图11.20等实例细加观察不难发现, 按照自左向右的次序, 任何两次相邻比对所对应的子串之间存在极强的相关性, 子串的指纹亦是如此。

实际上, 二者仅在首、末字符处有所出入。准确地如图11.21所示, 前一子串删除首字符之后的后缀, 与后一子串删除末字符之后的前缀完全相同。

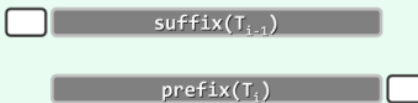


图11.21 相邻子串内容及指纹的相关性

利用这种相关性, 可以根据前一子串的指纹, 在常数时间内得到后一子串的指纹。也就是说, 整个算法过程中消耗于子串指纹计算的时间, 平均每次仅为 $O(1)$ 。

该算法的具体实现, 如代码11.12所示。

```
1 // 子串指纹快速更新算法
2 void updateHash ( HashCode& hashT, char* T, size_t m, size_t k, HashCode Dm ) {
3     hashT = ( hashT - DIGIT ( T, k - 1 ) * Dm ) % M; //在前一指纹基础上, 去除首位T[k - 1]
4     hashT = ( hashT * R + DIGIT ( T, k + m - 1 ) ) % M; //添加末位T[k + m - 1]
5     if ( 0 > hashT ) hashT += M; //确保散列码落在合法区间内
6 }
```

快速指纹计算

1188

❖ hash() 的计算, 似乎每次均需 $O(|P|)$ 时间

有可能加速吗?

- suffix(T_{i-1})

❖ 回忆一下, 进制转换算法...

❖ 观察

prefix(T_i) +

- 相邻的两次散列之间, 存在着某种相关性

- 相邻的两个指纹之间, 也有着某种相关性

❖ 利用上述性质, 即可在 $O(1)$ 时间内, 由上一指纹得到下一指纹...

11. 设文本串 T , 模式串 P 的长度分别为 2023 和 17, 且 T 并不包含 P , 借助 Boyer-Moore 算法的坏字符表 bc (好后缀表 gs 不用) 来做模式匹配, 最好情况下只需做 119 次字符比对。(2023)

解析:

· $O(n/m)$ —— 除法? 没错! 比如:

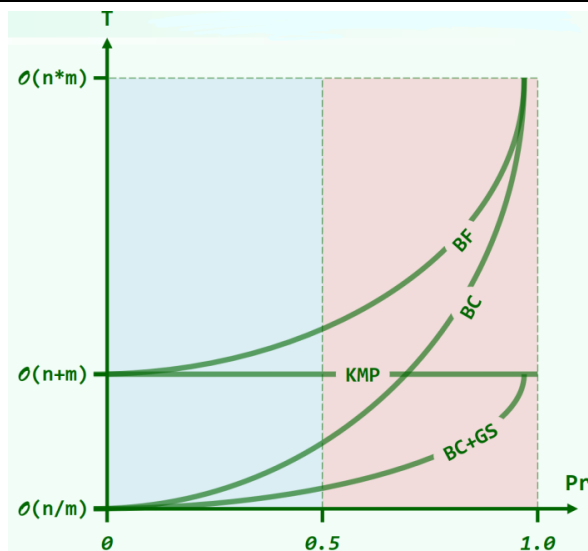
$$T = \begin{bmatrix} x & x & x & x & 1 \end{bmatrix} \begin{bmatrix} x & x & x & x & 1 \end{bmatrix} \begin{bmatrix} x & x & x & x & 1 \end{bmatrix} \begin{bmatrix} x & x & x & x & 1 \end{bmatrix} \begin{bmatrix} x & x & x & x & 1 \end{bmatrix}$$

$$P = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

本章真题总结:

串这里主要题型有以下几种:

一是考 kmp 算法, bf 算法, bm 算法的时间复杂度的分析, 也就是那个 ppt 上面的时间复杂度的图和习题 11-9 和习题 11-10 的两个结论。



二是考没改进的 next 的 kmp 算法的时间复杂度问题.

三是考 next 表和改进版的 next 表的构造.

四是考 gs 表的含义 (2018) 和仅用 gs 表下的 bm 算法的最好的时间复杂度 (2022)

五是考了 kr 算法快速指纹更新的结论.

六是考了仅 bc 的 bm 算法最少比对次数.

可能会考,但是真题尚未考过的题型 (涉及算法题的放到最后算法题的部分):

一是 next 表和 ss 表,gs 表构造的时间复杂度问题

二是 ss 表和 gs 表的构造

三是 01 串下 kmp 和仅 bc 表下的 bm 算法的时间的比较

四是因为什么不倾向于使用二维的 bc 表

[11-5] 针对坏字符在模式串 P 中位置太靠右,以至位移量为负的情况,11.4.2 节建议的处理方法是直接将 P 右移一个字符。然而如图 11.10(f) 所示,此后并不能保证原坏字符位置能够恢复匹配。为此,或许你会想到:可在 $P[j]$ 的左侧找到最靠右的字符 'X',并将其与原坏字符对齐。

a) 试具体实现这种处理方法;

【解答】

请读者按照以上思路,独立完成编码和调试任务。

b) 为什么我们不倾向于使用这种方法?

【解答】

尽管以上思路的实现方式可能不尽相同,但本质上都等效于将原先一维的 $bc[j]$ 表,替换为二维的 $bc[j][X]$ 表。具体地,这是一张 $m \times |\Sigma|$ 的表格,其中 $bc[j][X]$ 指向“在 $P[j]$ 左侧并与之最近的字符 'X'”。

如此,尽管预处理时间和所需空间的增长量并不大,但匹配算法的逻辑控制却进一步复杂化。最重要的是,此类二维 $bc[j][X]$ 表若能发挥作用,则当时的好后缀必然很长——此类情况,同时使用的 $gs[j]$ 表完全可以替代 $bc[j][X]$ 表。

五是 kr 算法冲突的概率 $1/M$,其中 M 是散列表长.

排序

- 1.折半插入算法在寻找插入的位置时采用二分查找,因此折半插入排序时间复杂度为 $O(n \log n)$ (×)(2016)

解析: 向量的话要移动,需要 $O(n)$ 时间; 列表的话,二分查找时间也是 $O(n)$

- 2.在进行起泡排序时,有可能出现某些元素在排序过程中一直远离它的最终位置 (√) (2016)

解析: 详见习题解析 1-3,比如 7,6,1,2,3,4,5 变成 6,1,2,3,4,5,7

如果纠结一直远离这四个字,可能得另做判断.

[1-3] 试分别举出实例说明,在对包含 n 个元素的序列做起泡排序的过程中,可能发生以下情况:

a) 任何元素都无需移动(从而内循环仅执行一轮即可终止算法);

【解答】

比如,所有元素已经按序排列。

b) 某元素会一度(朝着远离其最终位置的方向)逆向移动;

【解答】

比如序列:

$\{ n, \lfloor \frac{n-1}{2} \rfloor, 1, 2, \dots, n-2 \}$

经首轮扫描交换后为

$\{ \lfloor \frac{n-1}{2} \rfloor, 1, 2, \dots, n-2, n \}$

其中的次大元素 $n-1$, 最终位置在初始位置的右侧,在上述过程中却向左侧移动一个单元。

- 3.对序列 (64,63,...,2,1) 进行直接插入排序比较次数最接近于 () (2017)

(A)2800 (B)2600 (C)2400 (D)2200 (E)2000

解析: 比较次数 $= 1+2+3+\dots+63=2016$. 所以答案是 E

- 4.基于 CBA 的排序算法对所有大小为 n 的数组进行排序的时间复杂度是 $\Omega(n \lg n)$ (×) (2018)

解析: 插入和起泡最好情况都是 $O(n)$

- 5.基数排序的底层排序算法一定是稳定的 (√) (2018)

解析:

原始数据是: 23,21(a),21(b);

第一趟可能变成 21(b),21(a),23;

第二趟可能变成 23,21(b),21(a),会发生错误

- 6.交换序列中的任一对逆序元素,逆序对数必然减少。 (√) (2019)

解析:

设原本的序列为 $A_1 \dots A_i \dots A_j \dots A_n$, 其中 $A_i > A_j$, 交换了之后变成 $A_1 \dots A_j \dots A_i \dots A_n$ 原来的逆

序对 $A_i A_j$ 消失了.

考查 $A_1 \cdots A_i A_{i+1} \cdots A_{j-1} A_j A_{j+1} \cdots A_{n-1} A_n$ 变成 $A_1 \cdots A_j A_{i+1} \cdots A_{j-1} A_i A_{j+1} \cdots A_{n-1} A_n$ 之后的情况

[1] A_i 开头的逆序对个数的变化情况, 要么不变, 要么减少

[2] A_j 开头的逆序对个数的变化情况, 要么不变, 要么增多

情况[2]中如果逆序对个数增多, 因为 $A_i > A_j$, 所以也不至于比情况[1]中逆序对减少的数量要多. 又因为原来的逆序对 $A_i A_j$ 消失了, 所以逆序对个数必然减少.

比如 1, 8, 4, 6, 5, 3, 2 交换 8 和 5 变成了 1, 5, 4, 6, 8, 3, 2 原来 8 开头的逆序对个数是 4 个, 84, 86, 83, 82 交换了之后, 8 开头的逆序对个数是 2, 83, 82 减少了两个, 原来 5 开头的逆序对是 2 个, 53, 52 交换了之后, 5 开头的逆序对个数是 3 个 54, 53, 52, 增加了 1 个. 总体的逆序对显然减少了.

如果把 6 换成 0, 5 开头的逆序对就是增加的 2 个逆序对, 增加个数的不至于比减少的要多. 85 这个逆序对消失, 总体的逆序对减少.

7. 基数排序如果采用了不稳定的底层算法, 则输出序列未必正确. (✓) (2019)

解析:

原始数据是: 23, 21(a), 21(b);

第一趟可能变成 21(b), 21(a), 23;

第二趟可能变成 23, 21(b), 21(a), 会发生错误

8. 在插入排序过程中, 有序前缀每增加一个元素, 序列中的循环节个数即使不增加, 亦不致减少 (✗) (2019)

解析:

- ✦ 任何一个序列 $\mathcal{A}[0, n)$, 都可以分解为若干个循环节 // 设元素之间可定义次序
- ✦ 任何一个序列 $\mathcal{A}[0, n)$, 都对应于一个有序序列 $S[0, n)$ // 经排序之后
- ✦ 元素 $\mathcal{A}[k]$ 在 S 中对应的秩, 记作 $r(\mathcal{A}[k]) = r(k) \in [0, n)$
- ✦ 元素 $\mathcal{A}[k]$ 所属的循环节是:

$$\mathcal{A}[k], \mathcal{A}[r(k)], \mathcal{A}[r(r(k))], \mathcal{A}[r(r(r(k)))], \dots, \mathcal{A}[r(\dots(r(r(k)))\dots)] = \mathcal{A}[k]$$
- ✦ 每个循环节, 长度均不超过 n
- ✦ 循环节之间, 互不相交

$A[] = \{6, 5, 4, 3, 2, 1\}$

$S[] = \{1, 2, 3, 4, 5, 6\}$

$A[0]$ 对应的是 $S[5] \Rightarrow r(0) = 5$,

$A[5]$ 对应的是 $S[0] \Rightarrow r(5) = 0$, 有一个循环节: $6 \rightarrow 1 \rightarrow 6$

$A[1]$ 对应的是 $S[4] \Rightarrow r(1) = 4$,

$A[4]$ 对应的是 $S[1] \Rightarrow r(4) = 1$, 有一个循环节: $5 \rightarrow 2 \rightarrow 5$

$A[2]$ 对应的是 $S[3] \Rightarrow r(2) = 3$,

$A[3]$ 对应的是 $S[2] \Rightarrow r(3) = 2$, 有一个循环节: $4 \rightarrow 3 \rightarrow 4$

一共是 3 个循环节

进行一趟插入排序

$A[] = \{5, 6, 4, 3, 2, 1\}$

$S[] = \{1, 2, 3, 4, 5, 6\}$

$A[0]$ 对应的是 $S[4] \Rightarrow r(0) = 4$,

$A[4]$ 对应的是 $S[1] \Rightarrow r(4) = 1$,

$A[1]$ 对应的是 $S[5] \Rightarrow r(1) = 5$,

$A[5]$ 对应的是 $S[0] \Rightarrow r(5) = 0$, 有一个循环节: $5 \rightarrow 2 \rightarrow 6 \rightarrow 1 \rightarrow 5$

$A[2]$ 对应的是 $S[3] \Rightarrow r(2) = 3$,

$A[3]$ 对应的是 $S[2] \Rightarrow r(3) = 2$ 有一个循环节: $4 \rightarrow 3 \rightarrow 4$

一共是 2 个循环节, 所以循环节的数量少了

9. 相对于选择排序, 插入排序有什么优点?

解析: 最好情况的复杂度低, 为 $O(n)$, 插入排序是稳定的, 插入排序可以在线排序。

10. 快速排序平均时间复杂度为 $O(n \lg n)$, 最好时间复杂度也是 $O(n \lg n)$ (✓) (2020)

解析: 快速排序最好和平均情况下的时间复杂度都是 $O(n \lg n)$

11. 基于比较式算法可以在 $O(n)$ 内确定无序数据中的前 10% (✓) (2020)

解析: k 选取用 $O(n)$ 的时间, 然后再用 $O(n)$ 的时间确定前 10% 的数据

12. 底层算法不稳定情况下, 采用基数排序算法后 () (2020)

(A) 未必正确且不稳定

(B) 正确并且不稳定

(C) 未必正确并且稳定

(D) 正确并且稳定

解析:

原始数据是: 23,21(a),21(b);

第一趟可能变成 21(b),21(a),23;

第二趟可能变成 23,21(b),21(a),会发生错误

13.快速排序和插入排序 (7 分) (2021)

考察对 QuickSort 算法的如下调整: 任何子序列在规模缩减至 m 以下之后,便不再继续递归划分; 待所有递归都已终止后,再对整个序列调用一次 InsertionSort,如果待排序序列总长为 n ,那么 InsertionSort 需要花费多少时间? 为什么?

解析: 段间有序,段内无序,段长是 m ,每段的插入排序时间是 $O(m^2)$,一共是 n/m 段,所以总的时间复杂度的是 $O(nm)$, m 是常数级别的话,其实最后一趟插入排序的渐进时间复杂度就是 $O(n)$

14.希尔排序步长序列是 $\{0, 1, 3, 7, 15, \dots, 2^k-1, \dots\}$,时间复杂度是 $O(n^{3/2})$ (✓) (2022)

解析:

PS Sequence

1272

❖ Papernov & Stasevic, 1965 //also called Hibbard's sequence

$$\mathcal{H}_{PS} = \mathcal{H}_{Shell} - 1 = \{2^k - 1 \mid k \in \mathcal{N}\} = \{1, 3, 7, 15, 31, 63, 127, 255, \dots\}$$

❖ Different items MAY NOT be relatively prime, e.g., $h_{2k} = h_k \cdot (h_k + 2)$

But ADJACENT items MUST be, since $h_{k+1} - 2 \cdot h_k = 1$

❖ Shellsort with $\mathcal{H}_{ps}$ needs

- $O(\log n)$ outer iterations and

- $O(n^{3/2})$ time to sort a sequence of length n //Why ...

另外 2017 年考了一个新的归并排序,题目不完整,新的归并排序(题目不完整)

15.插入排序可实现就地(in-palce)算法 (✓) (2023)

解析:是,基本常识

本章真题总结:

排序这里主要题型有以下几种:

一是考基数排序底层排序算法必须是稳定的结论 (考了 2018,2019,2020)

二是考插入排序比较次数,折半插入排序的时间复杂度,插入排序和快速排序结合的算法的时间复杂度,插入排序和选择排序比较的优势.插入排序下循环节的个数的变化.

三是快排的平均和最好时间复杂度.

四是考 CBA 排序算法的时间复杂度

五是考了逆序对的变化,注意说的是逆序对还是相邻逆序对.

六是考了 top-k 算法的时间复杂度

七是考了基于 PS 序列的希尔排序的时间复杂度.

可能会考,但是真题尚未考过的题型(涉及算法题的放到最后算法题的部分):

一是全部的排序算法的最好,最差,平均时间复杂度,空间复杂度,以及排序的过程(这种题很可能会考,比如这个序列是哪个排序的第二趟排序的结果),归位不归位和其他特点(都可能会考),另外,希尔排序要结合步长序列考查.

Shell's Sequence 1261 ❖ Shell 1959: $\mathcal{H}_{shell} = \{1, 2, 4, 8, \dots, 2^k, \dots\}$ ❖ 实际上,采用 $\mathcal{H}_{shell}$, 在最坏情况下需要运行 $\Omega(n^2)$ 时间... ❖ 考查由子序列 $A = \text{unsort}[0, 2^{N-1}]$ 和 $B = \text{unsort}[2^{N-1}, 2^N]$ 交错而成的序列 ❖ 在做 2-sorting 时, A, B 各成一列; 故此必然各自有序 ❖ 然而其中的逆序对依然很多, 最后的 1-sorting 仍需 $1 + 2 + 3 + \dots + 2^{N-1} = \Omega(n^2/4)$ 时间 ❖ 根源在于, $\mathcal{H}_{shell}$ 中各项并不互素, 甚至相邻项也非互素	PS Sequence 1272 ❖ Papernov & Stasevic, 1965 //also called Hibbard's sequence $\mathcal{H}_{PS} = \mathcal{H}_{Shell} - 1 = \{2^k - 1 \mid k \in \mathcal{N}\} = \{1, 3, 7, 15, 31, 63, 127, 255, \dots\}$ ❖ Different items MAY NOT be relatively prime, e.g., $h_{2k} = h_k \cdot (h_k + 2)$ But ADJACENT items MUST be, since $h_{k+1} - 2 \cdot h_k \equiv 1$ ❖ Shellsort with $\mathcal{H}_{PS}$ needs - $\mathcal{O}(\log n)$ outer iterations and - $\mathcal{O}(n^{3/2})$ time to sort a sequence of length n //why ...	1274 $k \leq t$ 2) Consider those iterations for $\{h_k \mid k \leq t\} = \{\overleftarrow{h_1}, h_2, \dots, h_t\}$ ∴ h_{k+1} and h_{k+2} are relatively prime and are both in $\mathcal{O}(h_k)$ ∴ each h_k-sorting costs $\mathcal{O}(n \times h_k)$ time ∴ all these iterations cost $\mathcal{O}(n \times 2 \cdot h_t) = \mathcal{O}(n^{3/2})$ time ❖ This upper bound is TIGHT ❖ What about the average cases? - $\mathcal{O}(n^{5/4})$ based on simulations - but not proved yet $t < k$ $h_t < h_k$ $k = t$ $h_k = h_t$
Pratt's Sequence, 1971 1276 $\mathcal{H}_{pratt} = \{2^p \cdot 3^q \mid p, q \in \mathcal{N}\}$ $= \{1, 2, 3, 4, 6, 8, 9, 12, 16, 18, 24, 27, 32, 36, \dots\}$ ❖ Note that - adjacent items are NOT always relatively prime and - there are $\mathcal{O}(\log^2 n)$ items no greater than n ❖ With $\mathcal{H}_{pratt}$, Shellsort sorts a sequence of length n in $\mathcal{O}(n \cdot \log^2 n)$ time ...	Sedgewick's Sequence 1280 ❖ Pratt's sequence performs best asymptotically but - requires too many iterations and hence - is not good for pre-sorted sequences ❖ Sedgewick's sequence: a combination of PS's sequence with Pratt's $\{1, 5, 19, 41, 109, 209, 505, 929, 2161, 3905, 8929, 16001, 36289, 64769, \dots\}$ $\{9 \times 4^k - 9 \times 2^k + 1 \mid k \geq 0\} \cup \{4^k - 3 \times 2^k + 1 \mid k \geq 2\}$ - worst $\mathcal{O}(n^{4/3})$ & average $\mathcal{O}(n^{7/9})$ - best performance in practice ❖ Is there a step sequence with $\mathcal{O}(n \cdot \log n)$ worst-case performance?	

二是考一些组合的算法的特点和一些“新的”算法的特点.临场分析即可

三是考习题解析的一些小结论.

排序这一章的思维导图如下:



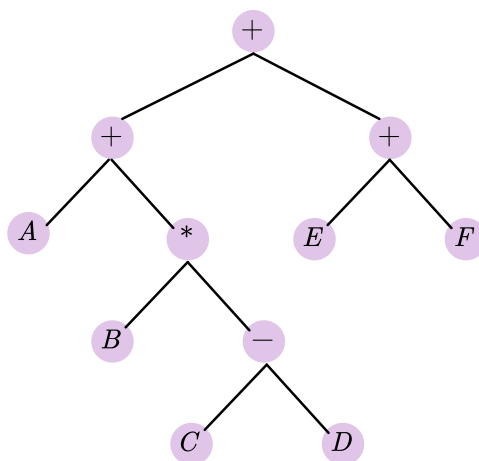
算法题

伪代码的语法

Todo

1.构造表达式树 (2016)

设计一个算法,把一个后序遍历 ABCD-*+EF??(后面三个符号忘记了不过不重要) 构造成如下图所示的二叉树。不妨设 ?? 为 ++



解析:

```

stack<tree> Num;
tree rebuild(string input)
{
    for(int i = 0; i < input.size(); ++i)
    {
        char tmp = input[i];
        if(tmp is 操作数)
        {
            tree NewTree(tmp); //用tmp初始化一棵新树
            Num.push(NewTree); //加入栈
        }
        else
        {
            tree rc = Num.pop(); //先出栈的是右子树
            tree lc = Num.pop(); //后出栈的是右子树
            attachAsRC(tmp, rc); //右子树接入
            attachAsLC(tmp, lc); //左子树接入
            Num.push(tmp); //再次入栈
        }
    }
    return Num.top(); //最后栈内就是表达式树
}
  
```

2.判断子数组 (2016)

求一个数组 A 中连续相同数字的和等于 s 的最长子数组长度,例如 $A=\{1,1,2,1,1,1,2,1\}$, $s=3$, 则所求子数组长度为 3,要求算法时间复杂度不超过 $O(n)$,空间复杂度不超过 $O(1)$

- 描述算法思想
- 伪代码实现
- 计算程序的算法复杂度。

解析:

a) 扫描一趟,遇到和等于 s 的子数组就计算子数组长度,更新最大值

b)

```
int seekMAX(int s, int A[])
{
    int k = 0
    int num = A[0]
    int count = A[0]
    int ans = 0;
    while(++k<A.size())
    {
        if(A[k] != num)
        {
            count = num = A[k];
        }
        else
        {
            count += A[k];
            if(count == s)
            {
                ans = max(ans, s/count);
            }
        }
    }
    return ans;
}
```

c) 一共扫描了一趟,时间复杂度是 $O(n)$

3.广搜最小环 (2017)

请利用图的广度优先遍历找出图中的最小环,若不存在环则输出+oo,要求时间复杂度为 $O(n \cdot e)$ 空间复杂度为 $O(n)$,最小环即环中边数最少的环。

- 请描述你的算法思想。
- 请用伪代码写出算法。
- 说明你的算法的时间复杂度和空间复杂度。

解析: 本题出自算法 (第四版) 练习题 4.1.17

4.1.17 图的周长为图中最短环的长度。如果是无环图, 则它的周长为无穷大。为 `GraphProperties` 添加一个方法 `girth()`, 返回图的周长。提示: 在每个顶点都进行广度优先搜索。含有 `s` 的最小环为 `s` 到某个顶点 `v` 的最短距离加上 `v` 到 `s` 的最短距离。

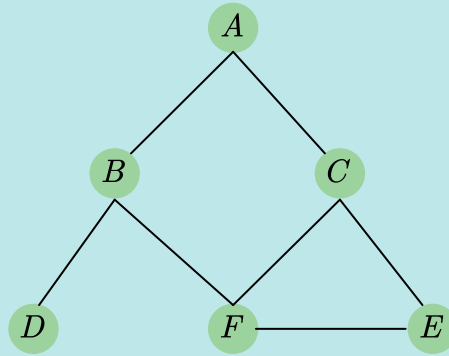
1) 以每一个顶点为起点做 BFS, 当遇到跨边的时候, 说明有环, 计算环路的边数, 更新最小环。

2)

```
int minCycle = INF; //初始化最小环边数为无穷大
for(int v = 0; v < Tv.size; v++)
{
    BFS(v);
}
//BFS的过程
void Graph<Tv, Te>::BFS(int v, int & clock)
{
    Queue<int> Q; //初始化队列
    status(v) = DISCOVERED; //v设置为DISCOVERED状态
    depth(v) = 0; //v是根节点, 深度是0
    Q.enqueue(v); //v进队
    while(!Q.empty()) //如果队列不空
    {
        int v = Q.dequeue(); //v出队
        dTime = ++clock;
        for(int u = firstNbr(v); -1 < u; u = nextNbr(v, u)) //枚举v的每一个邻居顶点u
        {
            if(UNDISCOVERED == status(u)) //如果u的状态是UNDISCOVERED
            {
                status(u) = DISCOVERED; //则把u的状态设置为DISCOVERED
                depth(u) = depth(v) + 1; //u的深度是v的深度加一
                Q.enqueue(u); //u进队
                type(v, u) = TREE; //v到u的边为树边
                parent(u) = v; //v是u的父亲
            }
            else
            {
                type(v, u) = CROSS; //v到u的边是跨边
                if(u != parent(v)) //如果u不是v的父亲
                {
                    minCycle = min{minCycle, depth(u) + depth(v) + 1}; //更新最小环的边数
                }
            }
        }
        status(v) = VISITED;
    }
}
```

3) 做了 n 次 BFS 每次 BFS 的时间复杂度是 $O(n+e)$, 总体时间复杂度是 $O(n(n+e))=O(ne)$, 只是申请了一个辅助队列, 空间复杂度是 $O(n)$

下面举例说明算法是怎么执行的, 例子在视频中会有讲解。



4. 二叉树的遍历 (2017)

若二叉树的数据结构如下

```

struct binarytree
{
    struct binarytree *parent;
    struct binarytree *lc;
    struct binarytree *rc;
    struct binarytree *first()
}

struct realbinarytree
{
    struct binarytree p;
    struct binarytree *next();
}
  
```

- (1) 若 first() 函数是取二叉树后序遍历节点的第一个节点, 请写出 first() 函数代码。
- (2) 若 next() 函数是取该节点的后序遍历的后继, 请写出 next() 函数代码。
- (3) 在调用 first() 函数和 next() 函数对二叉树进行后序遍历时, 证明遍历时间复杂度为 $O(n)$ 。

解析:

(1) 分三种情况讨论:

- a. 如果该节点有左孩子, 则后续遍历第一个节点为左子树中最靠左的分支中深度最深的节点
- b. 如果该节点有右孩子, 则后续遍历第一个节点为右孩子中最靠左的分支中深度最深的节点
- c. 如果该节点只有根节点, 则后续遍历第一个节点为该根节点

代码如下:

```

binarytree* first()
{
    while(this->lc || this->rc) //优先遍历左子树，左子树为空时，才转到右子树
    {
        if(this->lc)
        {
            this = this->lc;
        }
        else
        {
            this = this->rc;
        }
    }
    return this;
}

```

(2)也分成三种情况:

- 如果当前结点无父亲,其实当前结点就是根节点,其后序遍历后继为空。
- 如果当前结点是左孩子,其后序遍历后继为以其兄弟结点为根节点的子树的后序遍历的第一个结点 (若有)
- 如果当前结点是右孩子,其后序遍历后继为其父亲

代码如下:

```

binarytree* realbinarytree::next()
{
    if(!this->parent)
    {
        return NULL; //后继为空
    }
    if(this == this->parent->lc && !this->parent->rc) //当前结点是左孩子并且其右兄弟不为空
    {
        return this->parent->rc->first();
    }
    else
    {
        return this->parent; //是右孩子或者是左孩子但右兄弟为空的情况
    }
}

```

(3)

```

void PostTravel(realbinarytree* root)
{
    x = root->first();
    while(!x)
    {
        visit(x);
        x = x->next();
    }
}

```

树的每条边最多经过了两次,从父亲到自己一次,从自己到父亲一次。总体的时间复杂度就是 $O(2n)=O(n)$

5.单峰向量 (2018)

单峰向量定义为 $A[0,n]$,其中前缀严格递增 $A[0,k]$,后缀 $A[k,n]$ 严格递减。

- 1) 设计算法在 $O(\lg n)$ 的时间内找到最大值所在位置 k 。
- 2) 证明你算法的正确性。
- 3) 证明即使在最坏的情况下,你的算法复杂度也不会超过。

解析:

看 $A[mi], A[mi-1]$ 的 $A[mi+1]$ 大小关系,然后决定是直接返回

还是去左边找($A[mi] < A[mi-1] \&\& A[mi] > A[mi+1]$)

还是去右边找($(A[mi] > A[mi-1] \&\& A[mi] < A[mi+1])$),

直到向量长度小于 3 的时候,我们做最平凡的判断。

算法如下:

```
int search(int A[], int lo, int hi)
{
    int mi = (lo + hi) / 2;
    if (hi - lo < 3)
    {
        return mi = A[mi - 1] < A[mi] ? mi : mi - 1;
    }
    if (A[mi] > A[mi - 1] && A[mi] > A[mi + 1])
    {
        return mi;
    }
    if (A[mi] > A[mi - 1] && A[mi] < A[mi + 1])
    {
        return search(A[], mi + 1, hi);
    }
    if (A[mi] < A[mi - 1] && A[mi] > A[mi + 1])
    {
        return search(A[], lo, mi);
    }
}
```

因为是单峰向量,所以我们发现 $A[mi]$ 比左边的大,比右边的小的时候,一定是上升阶段,峰值在向量的右边,反之峰值就在向量的左边。

每次问题规模减少一半,总体的时间复杂度就是 $O(\lg n)$

6.连续子序列的最大和 (2018)

求一个向量 $A[0, n)$ 中最大的连续子序列的和。

- 1) 说明你的算法。
- 2) 使用伪代码描述你的算法。
- 3) 分析你算法的空间和时间复杂度。

(注: 你的得分取决于你的算法性能,性能在 $O(n^2)$ 以内的才能得分,时间复杂度 $O(n)$ 的才有可能得满分,蛮力算法不得分,因为其时间复杂度为 $O(n^3)$)

例如: 输入的数组为 1, -2, 3, 10, -4, 7, 2, -5, 和最大的子数组为 3, 10, -4, 7, 2, 因此输出为该子数组的和 18。

解析:

(1) 采用动态规划的思想,从最左端开始,一直扫描到最右端,最大值初始化为 0。如果和大于最大值,则更新最大值。如果和小于零,则从这个元素开始重新计算。

(2) 代码 (当然这个并不是伪代码。。。)

```
int MaxSumOfSeq(int num[],int n)
{
    int sum = 0;
    int b = 0;
    for(int i = 0; i < n; i++)
    {
        if(b < 0)
        {
            b = num[i];
        }
        else
        {
            b += num[i];
        }
        if(sum < b)
        {
            sum = b;
        }
    }
    return sum;
}
```

(3) 只扫描了一趟,时间复杂度是 $O(n)$,空间复杂度是 $O(1)$

减治策略：实现

71

❖ int gs_LS(int A[], int n) { //Linear Scan: $O(n)$

int gs = A[0], s = 0, i = n, j = n;

while (0 < i--) { //对于当前区间[i,j)

s += A[i]; //递增地得到其总和 : $O(1)$

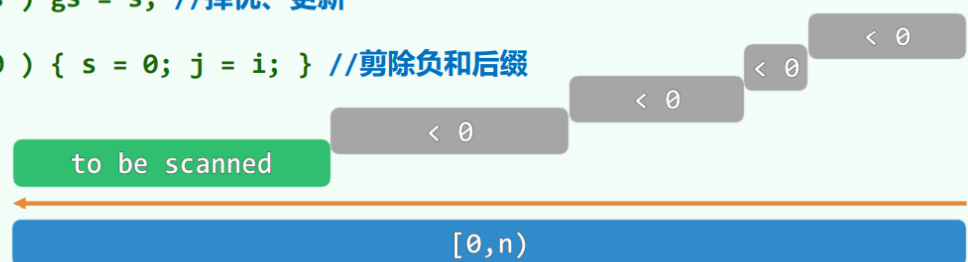
if (gs < s) gs = s; //择优、更新

if (s <= 0) { s = 0; j = i; } //剪除负和后缀

}

return gs;

}



7.后序遍历的第 K 个节点 (7+ 4+ 3= 14 分) (2019)

```
typedef struct BinNode {
    int size; //当前节点和孩子总数
    struct BinNode *lc, *rc; //左右孩子
}BinNode;

BinNode* rank(BinNode* t, unsigned int k) {
    // 0 <= k <= t.size
    //在此处补充不超过12行的代码
    //rank(t,k)实现以 t为根的二叉树中按照后序找出返回第k个节点,
    //若目标节点为x, 则计算时间空间复杂度均不可以超过O(depth(x)),即节点深度
    //请不要直接模拟后序遍历, 复杂度会超过要求, 将不得分
}
```

- (1) 给出具体实现算法
- (2) 说明原理 (200 字以内,可附一张插图)
- (3) 证明时间空间复杂度 (不超过 120 字)

解析:

```
BinNode *rank(BinNode* t, int k){
    if(t->size == k) return t;
    int num = 0;
    if(t->lchild != NULL) num = t->lc->size;
    if(num < k)
        return rank(t->rc, k - num);
    else
        return rank(t->lc, k);
}
```

如果 size 等于 k,那么此时的节点就是我们要找的节点,

否则递归地向下找,

左孩子的 size 比 k 小就去右孩子里面找,

如果左孩子的 size 比 k 大,就去左孩子里面找。

线性递归,每次递归深度加一,所以时间复杂度和空间复杂度都是 x 节点的深度。

8.算法题 (2020)

A,B,C,D 和 F 都是 3 分每题,E 为 5 分。下图给出二叉树的结构体声明:

```
struct BinNode { //二叉树节点
    BinNodePosi(T) lc;
    BinNodePosi(T) rc;
    BinNodePosi(T) parent;
    int size;
}
```

A.完全二叉树左子树的规模为_____,请给出递推公式;

B.给出 A 的伪代码实现 lSize();

C.search(k)找出中序遍历序列第 k 个节点 (中序遍历起始下标为 0,不要尝试模拟中序遍历,模拟中序遍历不得分) ;

D.splyto(a, x) ,a 为 x 的祖先,通过 zigzag 操作将 x 调整变为 a 的孩子,若 a 为 NULL,x 调为根节点;

E.将一颗 splay 树调成完全二叉树 (可用上述函数),要求时间复杂度为 $O(n \lg n)$,迭代深度不超过 $O(\lg n)$

F.证明你在 E 中给出的算法满足复杂度和迭代深度的要求

A.完全二叉树左子树的规模为_____,请给出递推公式;

解析: 设完全二叉树的规模为 n,其高度 $h = \lceil \log_2 n \rceil$

高度为 h-1 的满二叉树节点个数为: $2^h - 1$

此二叉树的最后一层节点个数为: $sum_last = n - (2^h - 1)$

高度为 h 的满二叉树最后一层节点个数为: $sum_full_last = 2^h$

完全二叉树左子树的规模: $Lsize = 2^{h-1} - 1 + \min\{sum_last, \frac{sum_full_last}{2}\} = 2^{h-1} - 1 + \min\{n - 2^h + 1, 2^{h-1}\}$

B.给出 A 的伪代码实现 lSize();

解析:

```
int LSize(int n) {
    int h = log(n) / log(2);
    int b = pow(2, h);
    int a = n - b + 1;
    int Lsize = b / 2 - 1 + min(a, b / 2);
    return LSize;
}
```

C.search(k)找出中序遍历序列第 k 个节点 (中序遍历起始下标为 0,模拟中序遍历不得分) ;

解析:


```

BinNodePosi(T) searchK( BinNodePosi(T) root, int k)
{
    int leftSize = root->lc->size;
    if(leftSize + 1 < k)
    {
        return searchK(root->rc, k-leftSize-1);
    }
    else if(leftSize + 1 == k)
    {
        return root;
    }
    else
    {
        return searchK(root->lc, k);
    }
}

```

D. `splyto(a, x)`, `a` 为 `x` 的祖先, 通过 `zigzag` 操作将 `x` 调整变为 `a` 的孩子, 若 `a` 为 `NULL`, `x` 调为根节点;

解析: 此题不用双层伸展应该也可以

```

void splyto(BinNodePosi(T) a, BinNodePosi(T) x)
{
    if(a == x->parent) return; //x已经使a的孩子了, 什么都不需要做
    else if(a == x->parent->parent)
    { //x是a的孙子, 只需一次旋转
        if(isLchild(x)) zig(x->parent);
        else zag(x->parent);
    }
    else
    { //x不是a的直接孙子
        BinNodePosi(T) p = x->parent;
        BinNodePosi(T) g = p->parent;

        if(isLchild(p) && isLchild(x)) { zig(g); zig(p); }
        else if(isRchild(p) && isRchild(x)) { zag(g); zag(p); }
        else if(isLchild(p) && isRchild(x)) { zag(p); zig(g); }
        else if(isRchild(p) && isLchild(x)) { zig(p); zag(g); }

        splyto(a, x); //继续检查a和x的关系, 做递归的旋转
    }
}

```

E. 将一颗 `splay` 树调成完全二叉树 (可用上述函数), 要求时间复杂度为 $O(n \lg n)$, 迭代深度不超过 $O(\lg n)$

解析:

```

BinNodePosi(T) to_Compl_BT(BinNodePosi(T) root)
{
    if(!root) return NULL;
    if(!root.lc && !root.rc) return root;
    int leftSize = lSize(root->size);
    BinNodePosi(T) newRoot = searchK(root, leftSize+1); //找到适合做完全二叉树根节点的结点
    splyto(root->parent, newRoot); //将newRoot旋转为根节点, 取代root原来的位置, 作为原来root的父亲的孩子

    to_Compl_BT(newRoot->lc); //递归构建左子树也为一棵完全二叉树
    to_Compl_BT(newRoot->rc); //递归构建右子树也为一棵完全二叉树
    return newRoot;
}

```

F. 证明你在 E 中给出的算法满足复杂度和迭代深度的要求

解析:

`lSize` 函数的复杂度为 $O(1)$

searchK 函数的复杂度为 $O(\lg n)$

splyto 函数的复杂度为 $O(\lg n)$

所以 to_Compl_BT 函数单次迭代的复杂度为 $O(1) + O(\lg n) + O(\lg n) = O(\lg n)$

总计有 n 个结点需要迭代,故 to_Compl_BT 函数总体时间复杂度为 $n \times O(\lg n) = O(n \lg n)$

而二叉树高度 $h = O(\log n)$,故迭代深度不超过 $O(\log n)$ 。

9.算法设计与分析 (8+5+3=16 分) (2021)

设二叉搜索树(Binary Search Tree)的节点定义如下:

```
struct BstNode
{
    int key;           //关键码
    BstNode *parent;  //父节点
    BstNode *lc;      //左孩子
    BstNode *rc;      //右孩子
    int size;         //当前节点及其后代的总数
}
```

(2) 试用伪代码完成以下算法,并提供足够的注释说明:

```
int count(BstNode *T, int lo, int hi) //lo < hi
{
    //在以T为根的二叉搜索树中,统计出关键码在[lo,hi)区间内的节点总数
    //无论统计出的总数有多大,时间复杂度均不得超过 $O(h)$ ,其中h为全树高度
    //除了作为输入的这棵树,只能使用 $O(1)$ 的辅助空间
}
```

解析:

(2) 试说明算法的原理及正确性

解析:

(3) 试证明算法的时间,空间复杂度均符合要求

解析: